NanoFold: A Compute-Efficient Benchmark for Controlled Research on Protein Structure Models

Chris Hayduk* [1] **Krithik Ramesh*** [2]

Abstract

Open-source AlphaFold (AF)-style systems have rapidly advanced protein structure prediction, but isolating the architectural and training choices that drive these gains remains difficult: production-scale training is computationally prohibitive outside well-resourced labs, and public corpora contain biological dependencies that can confound reduced-scale studies. We introduce **NanoFold**, a compact fixed-data benchmark for AF-style training studies, paired with a reference codebase for controlled head-to-head comparison. NanoFold defines three tracks evaluated with sealed hidden-validation labels: a `limited` track for sample efficiency, a `research_large` track for testing whether early gains persist under further optimization, and an `unlimited` track for the best achievable performance under the fixed data budget. Splits are disjoint by MMseqs2 sequence cluster and PDB entry and stratified by structural metadata, yielding 10,000 training, 1,000 public-validation, and 1,000 sealed hidden-validation chains. We verify the construction using structural features, sequence-family coverage, protein foundation model embeddings, and a randomization study over 1,000 alternative splits, finding NanoFold compositionally balanced and embedding-space typical. Through controlled ablations, longer-budget training, and evaluation on a broader OpenProteinSet pool, we show that NanoFold is learnable but unsaturated, separates training primitives, scales predictably with budget, and retains signal across sequence clusters, enabling transparent and reproducible architectural research at compute-accessible scale.

*Equal contribution [1]OpenAI, San Francisco, CA, USA [2]Lyra Labs, Cambridge, MA, USA. Correspondence to: Chris Hayduk <chrishayduk@openai.com>, Krithik Ramesh <krithik@lyralabs.ai>.

1. Introduction

AF-style protein structure prediction has become a steady source of new publicly released systems, with OpenFold (Ahdritz et al., 2024), ESMFold (Lin et al., 2023), Boltz-1 and Boltz-2 (Wohlwend et al., 2024; Passaro et al., 2025), Chai-1 (Chai Discovery et al., 2024), Protenix (ByteDance AML AI4Science Team et al., 2025), and the ongoing OpenFold3 effort (The OpenFold3 Team, 2026) all building on AlphaFold2 and AlphaFold3 (Jumper et al., 2021; Abramson et al., 2024). These systems vary architecture, training curriculum, optimization, and data pipeline simultaneously, which makes it difficult to attribute empirical gains to specific design choices. The field is therefore accumulating systems faster than it accumulates controlled evidence about which choices generalize, even as protein structure prediction continues to anchor active programs in drug discovery (Sadybekov & Katritch, 2023), protein engineering (Huang et al., 2016; Watson et al., 2023), and structural biology (Baek et al., 2021; Jumper et al., 2021). Closing this gap requires controlled experimentation at a scale that many groups can actually access.

Two natural routes are full-scale reproduction and reduced-scale training, but both remain difficult in practice. Reproducing AF-style training from scratch is still expensive: OpenFold reports approximately 50,000 GPU hours for its AlphaFold2 reproduction (Ahdritz et al., 2024), ScaleFold reduces AF2 pretraining to roughly 10 hours of wall-clock time only by distributing across 2,080 NVIDIA H100 GPUs (Zhu et al., 2024), and production-scale biomolecular efforts such as NVIDIA's Proteina-Complexa report multi-stage runs spanning tens to hundreds of thousands of optimizer steps on 16–96 A100 80-GB GPUs (Didi et al., 2026). OpenFold3-preview2 likewise remains a prerelease system whose reported training used 256 NVIDIA H100 GPUs (The OpenFold3 Team, 2026), putting multi-run attribution studies out of reach for nearly all groups. The subsampling route fails for different reasons. Public AF-style corpora contain sequence-cluster homology, repeated constructs, alternate crystal forms, and same-experiment chains, so naive random splits can leak biological signal across train and evaluation sets. Data-pruning results also show that the best selection strategy depends sharply on the data regime,

Figure 1. **NanoFold overview.** NanoFold builds a leakage-controlled OpenProteinSet-derived source pool, stratifies the training, public-validation, and hidden-validation splits by structural metadata, and evaluates AF-style models in fixed-data tracks with sealed hidden evaluation.

retaining hard examples when data are abundant but easy examples when data are scarce (Sorscher et al., 2022). A useful reduced benchmark therefore needs leakage-controlled splits, distributional stratification, a sealed evaluation set, and a shared training implementation. No existing resource provides this combination for small-data, small-parameter AF-style training.

NanoFold is our attempt at this combined construction; Figure 1 summarizes the resulting workflow. It pairs a deliberately constructed 10,000-chain training corpus, drawn from OpenProteinSet (Ahdritz et al., 2023), with public and sealed evaluation splits, an automated FoldScore evaluation protocol, and a reference AF-style training codebase. This design also makes NanoFold a practical test bed for controlled AF-style model development. Rather than requiring weeks of dataset construction, model implementation, and evaluation engineering before any research signal can be observed, NanoFold presents researchers with a small but structured protein-folding problem whose data, training code, and scoring function are already fixed. Using this fixed setup, we show that AF-style training learns meaningful structural representations at parameter and data scales below those used in prior studies, that ablation behaviors at these scales recover findings observed in production-scale models, and that NanoFold-trained models retain signal across a broader OpenProteinSet evaluation pool.

Contributions. We summarize our contributions as follows.

- **A fixed-data benchmark for small-scale AF-style training.** NanoFold provides a leakage-aware benchmark for studying architectural and training choices in AF-style protein structure models. It combines a 10,000-chain training set, a 1,000-chain public-validation set, and a 1,000-chain sealed hidden-validation set with tracks spanning increasing compute budgets. It also includes a reference codebase of AF-style baselines and ablations, enabling controlled comparison under matched data and matched implementation.
- **A reusable split-construction protocol for structured biological data.** NanoFold enforces MMseqs2 (Steinegger & Söding, 2017) cluster and PDB-entry disjointness, stratifies by structural metadata from CATH (Sillitoe et al., 2021), SCOPe (Chandonia et al., 2022), and ECOD (Cheng et al., 2014), and verifies split quality through a randomization study over 1,000 alternative valid splits. This provides a template for constructing reduced biological datasets without collapsing into leakage-prone or distributionally biased subsamples.
- **Empirical evidence that reduced-scale AF-style training is scientifically informative.** We show that AF-style models learn meaningful structural representations at parameter and data scales below those used in prior studies, that ablation behaviors in this regime recover findings observed in production-scale models, and that NanoFold-trained model performance remains competitive at longer budgets and across a broader OpenProteinSet evaluation pool.

2. Related Work

Community-scale blind assessment is the dominant evaluation regime for protein structure prediction, with CASP (Kryshtafovych et al., 2023; 2026) on unsolved targets, CAMEO (Haas et al., 2018) on weekly PDB releases,

and PXMeter (Ma et al., 2025) providing unified evaluation of several released biomolecular predictors across curated biological-complex benchmarks. ProteinNet (AlQuraishi, 2019) provides earlier standardized data, and AlphaFold DB (Varadi et al., 2022) contributes predicted structures as a resource rather than a labeled benchmark. These resources evaluate trained models and address what gains have been achieved rather than what drives them. NanoFold inherits the sealed-test-set practice but redirects it toward attribution at scales where architectural studies are practical to run from scratch.

Public code, data, and model-weight releases have made AF-style training reproducible. OpenFold (Ahdritz et al., 2024) and OpenProteinSet (Ahdritz et al., 2023) provide an AlphaFold2 reproduction and redistributable corpus; Boltz-1 and Boltz-2 (Wohlwend et al., 2024; Passaro et al., 2025), Chai-1 (Chai Discovery et al., 2024), Protenix (ByteDance AML AI4Science Team et al., 2025), and OpenFold3 (The OpenFold3 Team, 2026) extend the open release model to AF3-style biomolecular modeling; and ScaleFold (Zhu et al., 2024) and MegaFold (La et al., 2025) reduce wall-clock training cost. None of these resources alone provides a controlled testbed for architectural study. NanoFold builds on this infrastructure but inverts the optimization target. Existing systems work to reduce the cost of a fixed recipe; NanoFold fixes the regime so different recipes can be compared.

The integrity of such comparisons depends on the data itself, where avoiding sequence and structural leakage between training and evaluation has been a long-standing concern. Sequence clustering (Fu et al., 2012; Steinegger & Söding, 2017) and time-based PDB-deposition cutoffs, used in AlphaFold2's CASP14-cutoff training (Jumper et al., 2021) and Protenix's September 30, 2021 wwPDB cutoff (ByteDance AML AI4Science Team et al., 2025), are standard tools. Adjacent benchmark efforts including the Therapeutic Data Commons (Huang et al., 2021), MoleculeNet (Wu et al., 2018), and the Open Graph Benchmark (Hu et al., 2020) provide curated datasets with explicit, task-appropriate split and evaluation protocols; within structural biology, however, splits are usually reported as design choices rather than verified statistically. NanoFold extends this with a verification protocol combining metadata-balance audits, ESMC-6B-based embedding coverage diagnostics, and a conditional randomization study against 1,000 alternative valid splits, on top of MMseqs2 cluster and PDB-entry disjointness. To our knowledge, this combination has not previously been applied to protein structure benchmark construction.

3. Dataset Construction

The aim of NanoFold's data construction is to ensure that comparisons across architectural and training choices reflect modeling differences rather than artifacts of the data split. Achieving this with PDB-derived chains is harder than it appears, because the chains in the pool form a structured population with homologs, repeated constructs, close mutants, duplicated families, alternate crystal forms, same-complex chains, and historically biased regions of structure space. Splits drawn without care can leak close relatives across train and evaluation, bias one split over another, or concentrate training on dense redundant pockets. NanoFold's response is a leakage-controlled, stratified split over a fixed eligible pool, with sealed evaluation and statistical verification.

3.1. Source pool and eligibility

All splits draw from a single eligible source pool, so that every participant trains and evaluates on the same processable universe of chains under the same tensor schema. The pool is assembled from public OpenFold/OpenProteinSet assets (Ahdritz et al., 2023; 2024) and RCSB mmCIF coordinates (wwPDB Consortium, 2019; Burley et al., 2021; Westbrook et al., 2022), with OpenProteinSet supplying chain metadata, duplicate-chain mappings, and precomputed `uniref90_hits.a3m` MSA assets derived from UniRef90 (Suzek et al., 2015); mmCIF files supplying atomic coordinates from which atom14 labels are constructed; and CATH, SCOPe, and ECOD annotations providing structural and domain classifications for downstream balancing (Cheng et al., 2014; Sillitoe et al., 2021; Chandonia et al., 2022). Locking these assets in advance is central to the benchmark, since it gives every participant identical evolutionary input and prevents the leaderboard from becoming a contest over external retrieval or database freshness. A fixed data loader then serializes feature tensors (residue identities, MSA rows, deletion features, residue indices, masks, and zero-template tensors with $T = 0$) and label tensors (Cα coordinates and masks, atom14 positions and masks, residue indices, and resolution) per eligible chain.

Eligibility filters confine the pool to chains where the modeling task itself is well-posed under shared resources. A chain enters the pool only if it satisfies the official processability and quality gates: presence in the OpenFold chain cache; length $40 \leq L \leq 256$; monomeric status; a sequence containing only the 20 standard amino acids; resolution at most 3.0 Å when reported; the required OpenFold feature assets; and successful atom14 projection (sufficient sequence identity, coverage, aligned fraction, and valid Cα support between the feature-side query sequence and the extracted mmCIF coordinates). These gates confine the task to a single-chain, short-to-medium-length regime where every

selected example can be downloaded, represented, trained on, and scored consistently.

3.2. Split construction

To control biological dependencies, NanoFold forms units rather than sampling chains directly. Let $\mathcal{C}$ denote the eligible chains. We build a graph $G = (\mathcal{C}, E)$ with edges connecting chains that share an MMseqs2 sequence cluster (30% identity, 80% coverage) or a PDB entry (Steinegger & Söding, 2017; wwPDB Consortium, 2019), and assign every chain in a connected component to the same split. This blocks leakage between close homologs through sequence clustering and between chains from the same experiment through PDB grouping, and prevents large duplicated families from receiving disproportionate influence.

Unit-level grouping prevents leakage but not distributional skew. A random allocation of units could still produce a hidden-validation set whose chains are systematically more beta-rich, longer, or lower-resolution than those in training, in which case the leaderboard would measure distribution shift rather than data-efficient geometry learning. We stratify units before allocation by assigning each unit a label

$$z(u) = (\text{secondary}(u), \text{domain}(u), \\ \text{length\_bin}(u), \text{resolution\_bin}(u))$$

derived from the official chain annotations; these fields serve as balancing axes only and are not exposed to models or used as scoring labels. The allocator then samples units proportionally across strata to produce splits of size $|T| =$ 10,000, $|V_{\text{pub}}| = 1{,}000$, and $|V_{\text{hid}}| = 1{,}000$.

Repeatedly reused validation sets eventually become sources of implicit training signal, so NanoFold separates public validation from sealed hidden validation. The public-training and public-validation manifests are committed and hash-locked, and the hidden-validation split is generated privately after the public manifests are fixed: units containing any public-training or public-validation chain are removed, hidden counts are matched to the public-training and public-validation distributions, and hidden chains are selected deterministically from a private salt. During hidden evaluation the runtime serves only feature tensors and withholds Cα coordinates, atom14 labels and masks, and resolution, and hidden manifests, labels, features, fingerprints, and locks remain maintainer-only.

3.3. Statistical verification

Even with leakage controls and stratification in place, the committed split is one of many possible valid splits drawn under the same constraints, and its specific behavior is an empirical question. Three diagnostics establish its key properties: a metadata-balance audit checks that the splits are compositionally comparable with respect to the design fields; an embedding-space coverage analysis checks that training broadly covers the eligible pool without concentrating on dense redundant pockets; and a conditional randomization test checks how the committed split compares with valid splits drawn under the same constraints.

Table 1. **Split-comparability diagnostics.** Maxima across secondary-structure class, domain-architecture class, length bin, and resolution bin.

Comparison	Max JSD ($\times 10^{-3}$)	Max TV ($\times 10^{-3}$)	Max bin deviation ($\times 10^{-3}$)
Train vs. source pool	0.012	0.68	0.48
Public val. vs. source pool	0.68	9.6	6.8
Hidden val. vs. source pool	0.44	4.4	3.4
Train vs. public val.	0.77	9.1	6.5
Train vs. hidden val.	0.57	4.6	3.1
Public vs. hidden val.	0.56	11.0	7.0

Metadata balance. For each categorical balancing field b, let p_S^b denote the empirical distribution of split S over the buckets of b, and $p_{\mathcal{U}}^b$ the corresponding distribution over the eligible disjoint-unit source pool. We measure split discrepancy using Jensen–Shannon divergence (JSD), total variation distance (TV), and maximum bin deviation (Appendix A.1.1). Table 1 reports the maximum discrepancy across the four audited fields; the largest pairwise JSD, at 7.66×10^{-4} (train vs. public validation), and the largest pairwise maximum bin deviation, at 0.70 percentage points, are both small. The hidden-validation split is sealed for ranking and matched to the public regime over the metadata axes that most directly affect difficulty and label quality.

Embedding-space coverage. Metadata bins capture only commonly tracked coarse structural features, and two splits could match on secondary structure, length, and resolution while differing in the fine-grained sequence neighborhood structure that determines what a model actually sees during training. We embed each eligible chain with the 6B-parameter ESMC variant (Candido et al., 2026) as a mean-pooled vector $\phi(x) \in \mathbb{R}^{2560}$ and compare via cosine distance. Public and hidden validation are tightly matched in this space (Figure 2), with median nearest-train distances of 0.0847 and 0.0843, and 99.0% and 99.0% of examples lying within their local 50-neighbor radius of a training example; both sit closer to training than the broader non-train pool. Training is also not allocated to match raw PDB row frequencies: the highest-density decile of the source pool receives only 2.25% of training chains, because the goal is broad coverage under strict leakage controls rather than reproduction of biological deposition history. Quantile breakdowns and coarse-cluster occupancy at $K = 100$ and $K = 200$ appear in Tables 3 and 4.

Conditional randomization. The committed split is fi-

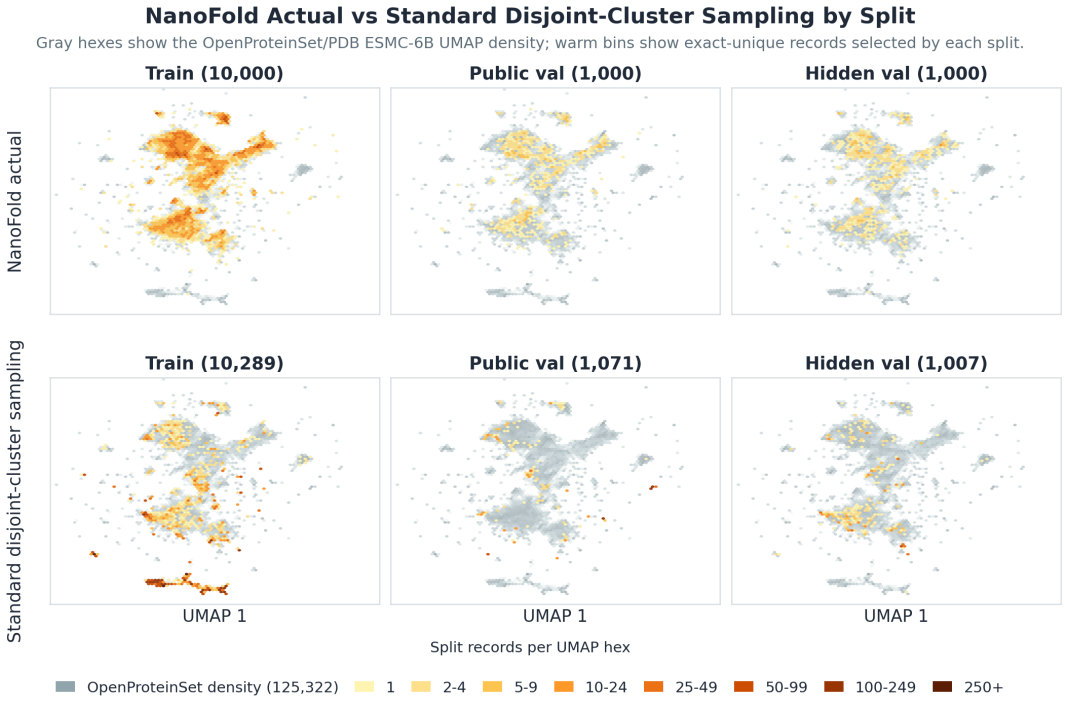


Figure 2. **Embedding-space coverage.** UMAP projection (McInnes et al., 2018) of 2,560-dimensional ESMC 6B mean embeddings (Candido et al., 2026) over the eligible OpenProteinSet-derived source pool. Rows compare NanoFold's actual split allocator (top) with an approximately equal-sized allocation constructed by sampling whole MMseqs2 clusters (bottom); the three columns show the training, public-validation, and hidden-validation sets (NanoFold: 10,000/1,000/1,000 chains; standard sampling: 10,289/1,071/1,007). Across all three sets, NanoFold's sampling approach provides broader coverage of the OpenProteinSet universe, while cluster sampling is concentrated in high-density regions. Quantitative coverage is reported in Tables 3 and 4.

nally compared against $B = 1{,}000$ alternative valid splits drawn under the same official constraints (cluster disjointness, PDB-entry disjointness, sizes, reserve, stratification). Embedding-space diagnostics lie inside the central randomized range, while public max JSD is above the randomized p95 but remains tiny in absolute terms (0.703×10^{-3}). In ESMC space, public validation remains slightly closer to training than the randomized median, with nearest-train p50 of 0.0847 versus randomized 0.0849, and public-validation coverage at the global median 50-neighbor radius is 0.7352 versus randomized 0.7281. The $K = 100$ touched-cluster mass is also central, 0.8741 versus randomized 0.8808. The full comparison is in Appendix A.1.2.

Together these diagnostics support the core data claim: training, public-validation, and hidden-validation are compositionally matched over biologically relevant metadata, training broadly covers the eligible pool in embedding space, and the committed split's embedding-space diagnostics behave like a typical draw from the official allocation procedure.

4. The NanoFold Benchmark

A fixed dataset becomes a benchmark only when paired with an evaluation protocol that asks well-posed questions. NanoFold asks three: which methods are most data efficient, whether their early advantages persist under additional optimization, and what the best achievable accuracy is on the fixed corpus. Each is a separate track over the same training task, training rules, and scoring metric, differing only in compute budget and ranking rule.

4.1. Task and prediction contract

The task is to predict atom14 coordinates from sequence and the official MSA-derived features. Each chain x has length L_x, residue identities $a_{1:L_x}$, MSA features m_x, and atom14 supervision

$$y_x = \{Y_x, M_x\},$$
$$Y_x \in \mathbb{R}^{L_x \times 14 \times 3},$$
$$M_x \in \{0,1\}^{L_x \times 14}$$

where Y_x contains atom14 coordinates and M_x is the resolved-atom mask. A model f_θ takes the feature-side tensors and returns $\widehat{Y}_x = f_\theta(a_{1:L_x}, m_x) \in \mathbb{R}^{L_x \times 14 \times 3}$, supplied to the scorer as a single floating-point tensor `pred_atom14` of shape $(B, L, 14, 3)$ per batch.

4.2. Tracks and training rules

The three tracks are summarized in Table 2. The fixed-budget tracks (`limited` at 240,000 samples and `research_large` at 960,000) rank submissions by the area under the hidden FoldScore learning curve (AUC), rewarding methods that acquire useful geometry early; the `unlimited` track ranks by final hidden FoldScore. Official ranking is performed on the sealed hidden-validation split, with the public-validation split reserved for development.

Table 2. **NanoFold tracks.** All tracks share the same training corpus, validation splits, prediction contract, and training rules. Fixed-budget tracks are ranked by hidden FoldScore AUC to reward early acquisition of useful geometry; the unlimited track is ranked by final hidden FoldScore.

Track	Sample budget	Rank metric	Scientific question
`limited`	240,000	Hidden FoldScore AUC	Which methods learn fastest?
`research_large`	960,000	Hidden FoldScore AUC	Do gains persist with more optimization?
`unlimited`	Unrestricted	Final hidden FoldScore	What is the best fixed-data final performance?

The training rules keep the experimental object narrow. Official templates are disabled, external structure data is disallowed, external MSA generation is excluded from official runs, and models are trained from scratch on the fixed public-training set. These rules isolate which architectures, losses, curricula, and biological priors most efficiently acquire transferable protein geometry under a small, fixed set of experimentally derived structures.

4.3. Scoring

FoldScore is adapted from the CASP15 tertiary-structure scoring framework (Simpkin et al., 2023), combining four families of measurements that can all be computed from the official atom14 prediction contract: global Cα fold accuracy via GDT_HA (Zemla, 2003); local atomic agreement via lDDT (Mariani et al., 2013), CAD (Olechnovic et al., 2013), and side-group and side-chain accuracy (SG, SC); backbone and dihedral consistency (BB, DipDiff); and steric plausibility via MolProbity-style clash validation (Chen et al., 2010).

For a single chain,

$$\begin{aligned}\text{FoldScore} = {} & 0.25 \cdot \text{GDT\_HA}_{C\alpha} \\ & + 0.09375 \cdot (\text{lDDT}_{\text{atom14}} + \text{CAD}_{\text{aa}} \\ & \quad + \text{SG} + \text{SC}) \\ & + 0.125 \cdot (\text{Clash} + \text{BB} + \text{DipDiff}) .\end{aligned}$$

FoldScore introduces minor changes to CASP15's scoring in order to make it tractable for NanoFold. First, CASP15 ranks groups by per-target z-scores over the submitted model pool, with outlier removal and penalty clipping, rather than by an absolute per-model score. Such relative normalization would make a NanoFold score depend on the set of competing submissions and would change as new leaderboard entries or checkpoint curves are added. In addition, the CASP15 formula contains two terms outside NanoFold's prediction contract: ASE, which evaluates submitted per-residue confidence/pLDDT against observed local accuracy, and reLLG, which measures molecular-replacement utility through a specialized crystallographic assessment path. Requiring either would add submission channels or external scoring dependencies unrelated to the fixed atom14 coordinate task. FoldScore therefore keeps the CASP15 structure-derived components that are computable from `pred_atom14`, official residue identities, and atom14 masks, and renormalizes the CASP15 weights over that supported subset.

For fixed-budget tracks, models are evaluated at checkpoints indexed by cumulative samples $0 = s_1 < \cdots < s_K \leq B$, where B is the track budget. Letting F_k denote mean hidden FoldScore at checkpoint k,

$$\text{AUC}_{\text{FoldScore}} = \frac{1}{B} \sum_{k=1}^{K-1} \frac{F_k + F_{k+1}}{2} (s_{k+1} - s_k).$$

Final-score evaluation rewards models that eventually converge; the AUC ranking rewards methods that acquire structural quality early. Together with the sealed hidden-validation split and the shared training rules, this scoring makes the leaderboard interpretable as a measurement of sample-efficient geometry learning.

5. Experiments

We use NanoFold to ask whether a deliberately small, leakage-controlled benchmark can reveal the core architectural components and inductive priors that underlie strong protein-structure prediction performance. To support this, we start by running detailed architectural ablations on a small AlphaFold2 implementation. All runs train from scratch on the 10,000-chain public-training split and are evaluated on the 1,000-chain public-validation split. Unless otherwise stated, runs use an effective batch size of 8, train for 30,000 optimizer steps, corresponding to 240,000

training samples, and evaluate public-validation FoldScore every 3,000 optimizer steps. Full hyperparameters and run definitions are given in Appendix A.2.

Lastly, we introduce several experiments that test NanoFold's compact dataset as a proxy for the broader OpenProteinSet-derived protein distribution. Models trained on NanoFold using a fixed budget of unique chains outperform models trained on a standard OpenProteinSet subsample. At a larger compute budget, NanoFold-trained models remain competitive with models trained on the all-eligible OpenProteinSet-derived corpus, even though the broader corpus exposes them to 5.8 times as many unique chains.

Together, these results demonstrate that NanoFold is a compact dataset that captures generalizable signal and supports large-scale controlled ablations for discovering new biological priors.

5.1. NanoFold recovers AlphaFold2 scaling laws

The first requirement for a benchmark of this kind is calibration: models should improve as capacity and compute increase, but the task should not saturate so quickly that ablations collapse into noise. Figure 3 shows both effects. Increasing the minAlphaFold2 profile from tiny to medium to large improves final public-validation FoldScore from 0.355 to 0.457 to 0.649 under the same 240,000-sample budget. This is the expected direction: larger models can exploit the same fixed data more effectively.

The capacity ordering is also visible in individual structure predictions. Figure 4 compares the tiny, medium, and large profiles on five public-validation chains spanning the secondary-structure split buckets. Predictions are Kabsch-aligned (Kabsch, 1976) to the ground-truth structure using valid Cα atoms before rendering, with ground truth in gray and the tiny, medium, and large predictions in pink, blue, and green, respectively. We grouped chains using the split metadata and used PyMOL DSS assignments on the ground-truth PDBs to select examples whose rendered cartoons visibly matched each intended class. These examples give a qualitative view of how the aggregate scaling gains translate into recovered folds across distinct structural regimes, rather than an additional score-selected comparison.

NanoFold also exposes returns to effective depth at fixed parameter count. Holding the medium profile fixed and increasing the maximum number of recycling iterations improves final public-validation FoldScore from 0.457 at the default setting, which samples one or two training recycles and evaluates with two, to 0.472 with sampled training recycles up to eight. A fixed-four recycling ablation reaches 0.472 FoldScore. Thus, even in this compact regime, NanoFold distinguishes gains from parameter scale and gains from additional iterative refinement. This is useful

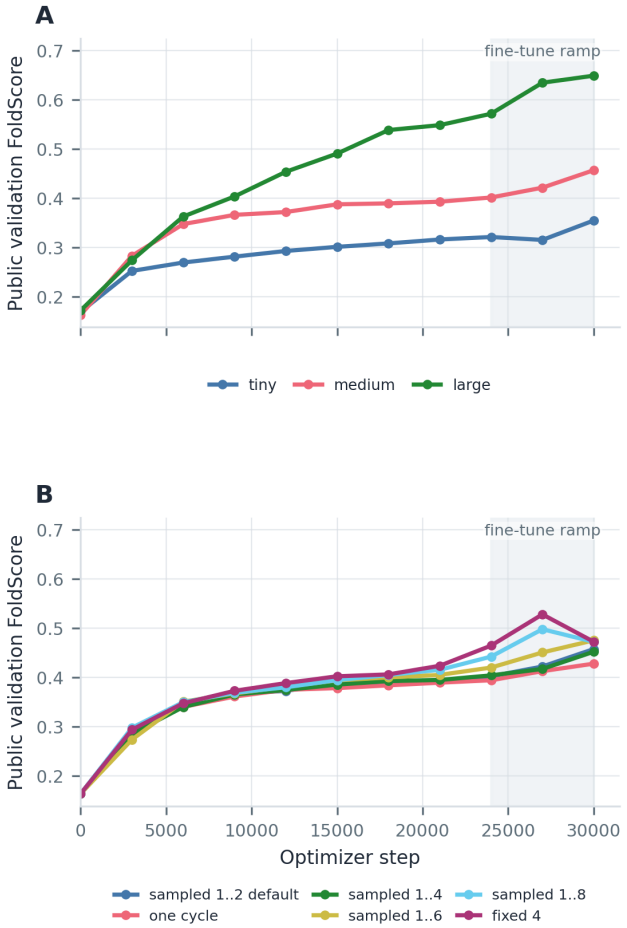


Figure 3. **Scaling and recycling on NanoFold.** Panel A shows that public-validation FoldScore improves as parameter count increases from tiny to medium to large, and Panel B shows gains from increasing the number of recycling iterations at fixed medium-model scale. Recycling increases effective model depth without changing the underlying parameter count. The shaded span marks the fine-tuning phase.

for model-development studies because it means that architectural and training choices can be compared before production-scale training is affordable.

5.2. Controlled ablations recover composable signals

A benchmark intended for architecture and training studies should do more than rank finished models; it should identify changes that can be recombined into better systems. We therefore ran medium-profile ablations over three training primitives suggested by AF-style practice and by our early NanoFold runs: recycling policy, backbone-FAPE clamping, and the late fine-tuning loss phase. Figure 5 shows a compact representative set of these runs alongside the FAPE clamping sweep.

Relative to the medium AF2-default recipe, fully unclamp-

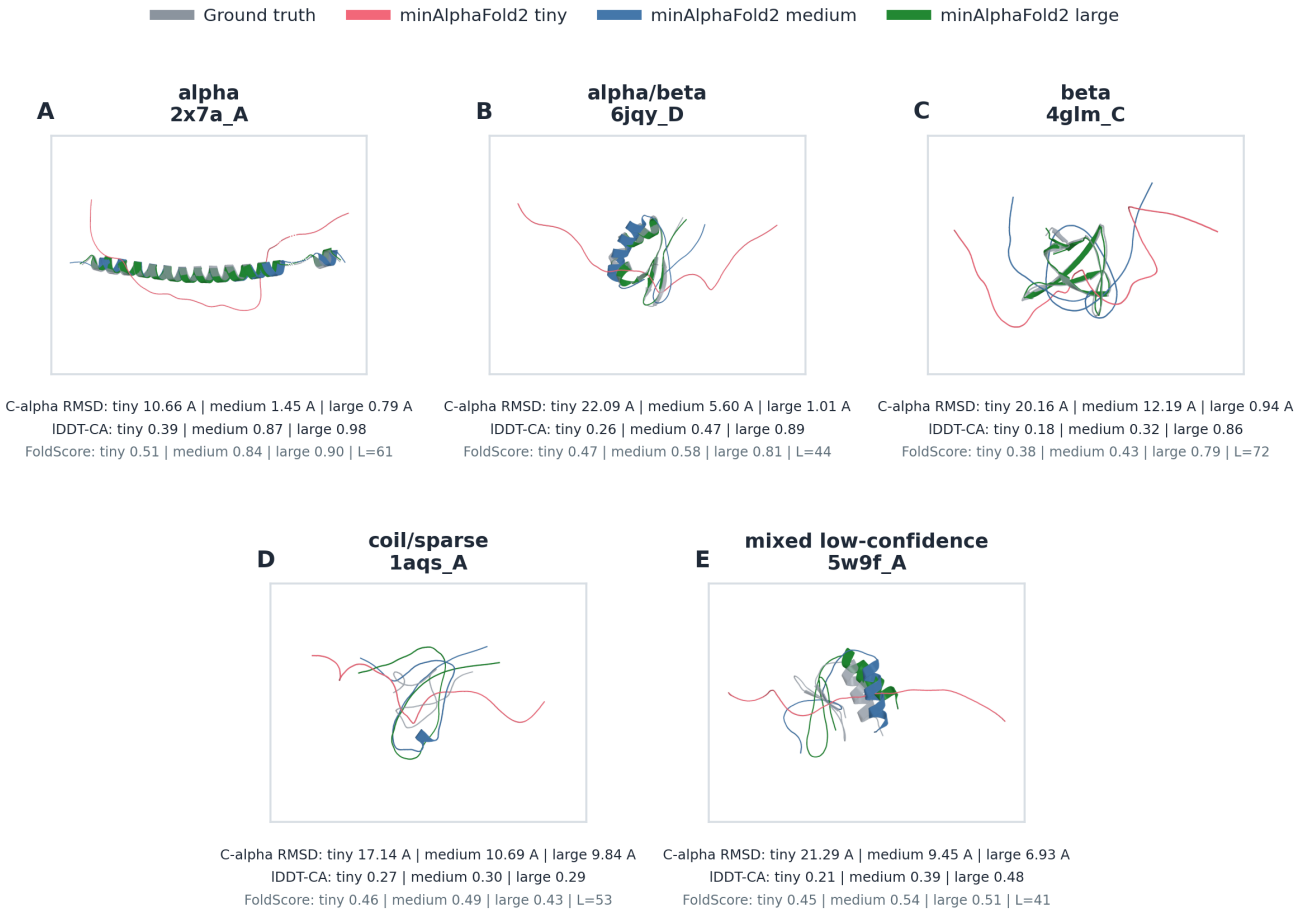


Figure 4. **Model scaling across representative public-validation structures.** Tiny (pink), medium (blue), and large (green) minAlphaFold2 predictions are aligned with ground truth (gray) for five chains selected to visually represent the secondary-structure split buckets: 2x7a_A (alpha), 6jqy_D (alpha/beta), 4glm_C (beta), 1aqs_A (coil/sparse), and 5w9f_A (mixed low-confidence). The overlays complement the aggregate capacity scaling in Figure 3.

ing backbone FAPE produces a clear improvement in final public-validation FoldScore (0.484 versus 0.457). Fixed-four recycling gives a smaller but directionally useful gain (0.472). Removing the fine-tuning loss phase alone lowers FoldScore to 0.400. This divergence reflects FoldScore's combination of global accuracy, local agreement, contact preservation, and physical plausibility.

Most importantly, these ablation signals can be composed to improve model performance. A model that combines fixed-four recycling, fully unclamped backbone FAPE, and no fine-tuning loss phase reaches 0.504 final public-validation FoldScore, making it the best medium-profile ablation in this study. This is the desired behavior for NanoFold as an experimental instrument: individual ablations provide information that is predictive enough to guide a follow-up configuration, rather than merely explaining results after the fact.

NanoFold also reproduces a known AF-style sensitivity to backbone-FAPE clamping policy at the 10,000-chain scale. AlphaFold2 uses a mixed scheme in which 90% of training mini-batches clamp backbone FAPE at 10 Å and 10% leave it unclamped (Jumper et al., 2021), and OpenFold later reported that samplewise rather than batchwise clamping improved convergence (Ahdritz et al., 2024); both observations were made in substantially larger regimes than NanoFold. Panel B of Figure 5 shows that the same sensitivity is visible here. The AF2-default batchwise 90/10 policy, samplewise 90/10, and samplewise 50/50 policies perform similarly by final FoldScore (0.457, 0.455, and 0.457). The deterministic soft 90/10 mixture and unclamping only after the fine-tuning boundary are lower (0.424 and 0.419), while fully unclamped backbone FAPE reaches the highest final FoldScore in the sweep (0.484). The curves separate throughout training rather than only at the final score, indicating that NanoFold is sensitive to the optimization dynamics induced by the loss itself, not just to its endpoint.

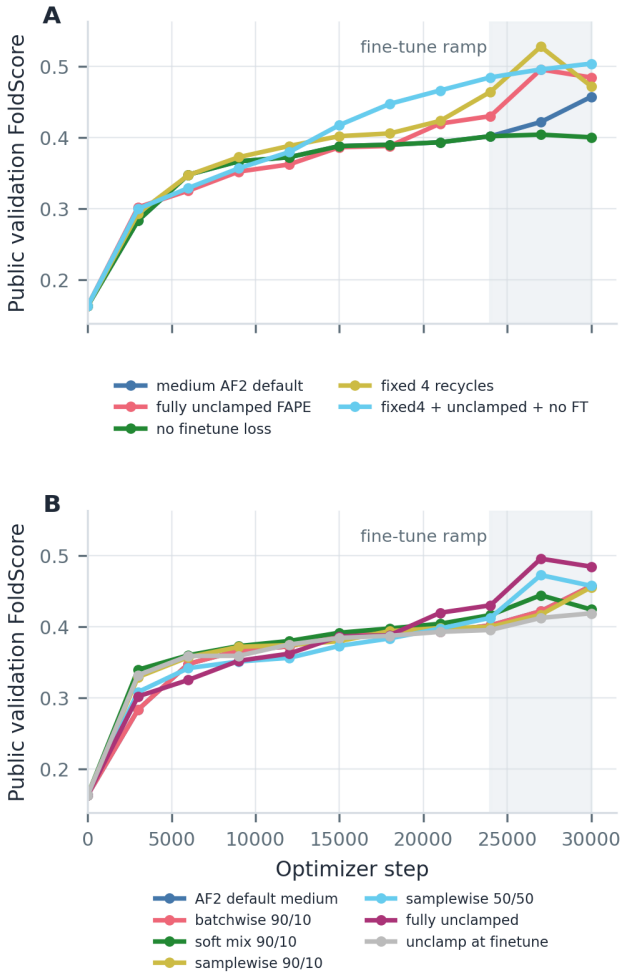


Figure 5. **NanoFold ablation signals.** Panel A shows representative medium-model ablations across recycling policy, backbone-FAPE clamping, and the late fine-tuning loss phase, with the best combined setting using fixed-four recycling, fully unclamped backbone FAPE, and the initial objective throughout training. Panel B shows that backbone-FAPE clamping policy strongly affects small-scale AF-style training. Public-validation curves compare AF2-default batchwise 90/10 clamping, deterministic soft mixing, samplewise clamping, delayed unclamping, and fully unclamped backbone FAPE. Shaded spans mark the fine-tuning phase.

5.3. NanoFold training remains competitive at longer budgets

The preceding experiments show that NanoFold separates architectural and optimization choices under a short, compute-accessible budget. We next asked whether this behavior persists when training is extended substantially beyond the default 30,000-step setting. In particular, we compared longer NanoFold training against longer training on a broader OpenProteinSet-derived corpus. This comparison probes whether NanoFold's deliberately compact, leakage-controlled 10,000-chain construction acts as a useful proxy for broader protein training, rather than merely defining an isolated small benchmark.

Figure 6 shows that extending training on NanoFold produces trajectories comparable to longer training on the broader corpus for the model profiles studied. Although the broader training set exposes the model to 5.8 times as many unique protein chains, the NanoFold-trained runs remain competitive over the same extended optimization horizon. This indicates that the compact split retains substantial learnable signal for these baseline models.

This result supports the dataset-construction strategy: density-aware, leakage-controlled sampling can preserve useful geometric diversity even under a severe data budget. It also motivates NanoFold as a lower-cost environment for generating and prioritizing model-design hypotheses.

5.4. NanoFold-trained models transfer to the broader OpenProteinSet

As an external check beyond the NanoFold public-validation split, we compared the NanoFold-trained AF2-medium model with a matched AF2-medium model trained in a standard OpenProteinSet-derived setup using the same unique-chain budget. Both models were evaluated on a large OpenProteinSet validation set that was mutually disjoint from both training sets. This comparison asks whether the signal learned from NanoFold's small, leakage-controlled training set carries over to a broader OpenProteinSet evaluation pool.

Figure 7 compares paired OpenProteinSet cluster means for FoldScore and lDDT-Cα. Each point or bin compares the same MMseqs2 cluster under the two training setups. The diagonal is the tie line: points above it indicate clusters where the NanoFold-trained model scores higher, while points below it favor the standard-trained model. Collapsing by MMseqs2 cluster reduces the influence of large, redundant protein families and aligns the comparison with the split discipline used throughout NanoFold.

The paired distributions show that the NanoFold-trained AF2-medium model improves performance across many OpenProteinSet clusters rather than concentrating its advantage in a small number of chains. It outperforms the standard-trained model in 66.6% of clusters by FoldScore and 72.3% by lDDT-Cα, demonstrating gains across diverse protein families and regions of protein-structure space.

6. Discussion

NanoFold addresses challenges in model development that are unique to the data-constrained regime characteristic of protein structure prediction. When experimentally resolved structures cannot be expanded on demand, progress depends increasingly on inductive biases that extract more transferable structural signal from each labeled example. Identify-

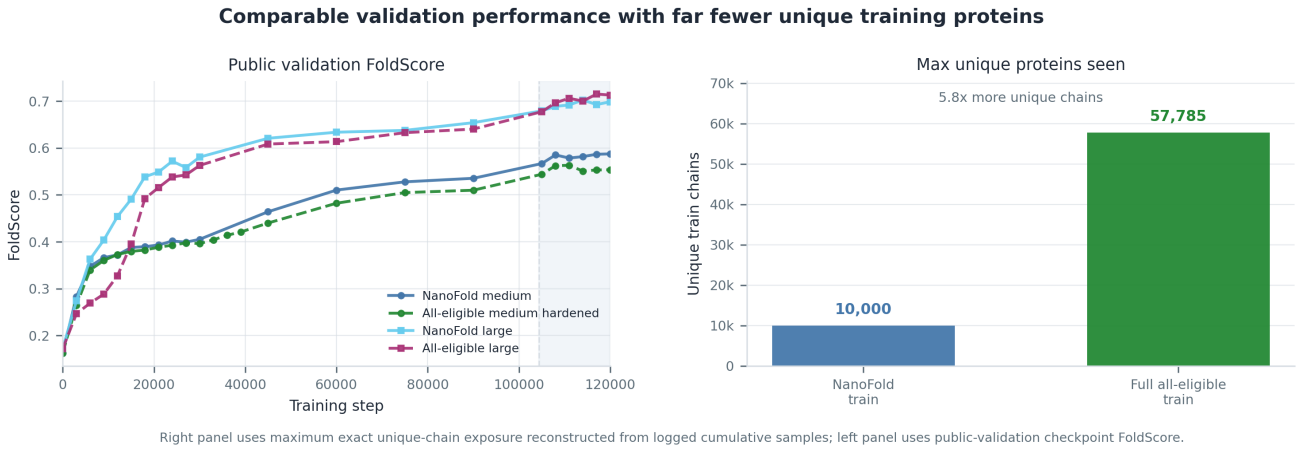


Figure 6. **Longer-budget NanoFold training remains competitive with far fewer unique training chains.** Left: public-validation FoldScore trajectories compare extended training on NanoFold with extended training on a broader OpenProteinSet-derived ("all-eligible") corpus at both medium and large model scale. Right: maximum number of unique training chains observed under the logged epoch-shuffled schedules; the all-eligible corpus exposes models to 57,785 unique training chains, 5.8 times as many as NanoFold's 10,000. NanoFold-trained runs remain competitive over the same 120,000-step optimization horizon. The shaded band marks the fine-tuning phase.

ing those biases requires suites of matched experiments, yet production-scale training makes such suites difficult to run. Further complicating matters, new models typically introduce changes to architecture, data, objectives, and optimization together, making it difficult to isolate which changes encode useful inductive biases. NanoFold holds the data and optimization factors fixed while reducing the cost of testing architectural changes. In the core study, the medium baseline requires 15.75 H100-hours (approximately $47), and the strongest combined medium-model ablation requires 23.38 H100-hours (approximately $70). This scale makes detailed searches over model priors accessible to community researchers within a bounded compute budget.

NanoFold is both compute-accessible and experimentally informative. Under fixed data and sample exposure, it recovers the expected ordering with model capacity and iterative recycling, separates the effects of geometric loss design and training curriculum, and returns signals that can be composed into a stronger model. Combining fixed-four recycling, fully unclamped backbone FAPE, and the initial loss throughout training raises the AF2-medium public-validation FoldScore from 0.457 to 0.504. Thus, NanoFold can expose meaningful inductive-bias and optimization signals without requiring full-scale training budgets.

The longer-budget and broader-data experiments provide complementary tests of whether findings from this compact regime extend beyond its public-validation split. Over 120,000 training steps, NanoFold-trained models remain competitive with models exposed to $57{,}785$ rather than $10{,}000$ unique chains. On the broader OpenProteinSet-derived evaluation pool, the NanoFold-trained model is favored on 66.6% of 9,763 MMseqs2 sequence clusters. These results support NanoFold as a compute-accessible environment for generating, rejecting, and prioritizing inductive-bias hypotheses that generalize to the broader space of protein structures.

NanoFold targets a defined slice of protein space: eligible monomeric OpenProteinSet-derived chains in the 40–256 residue range under the official MSA and chain-quality filters. The benchmark omits template ingestion, external MSA retrieval, multi-chain complexes, ligands, and conformational ensembles; CASP and production-scale benchmarks such as PXMeter provide complementary broader-scale evaluation. Public and sealed splits are comparable on the audited metadata. Natural extensions can measure additional biological axes through multi-chain and ligand variants, adversarial sealed splits, AF3-scale versions of the construction, and related benchmarks in which dependencies among biological examples make naive splitting unreliable. The training corpus, evaluation infrastructure, and reference codebase are publicly released to support these directions.

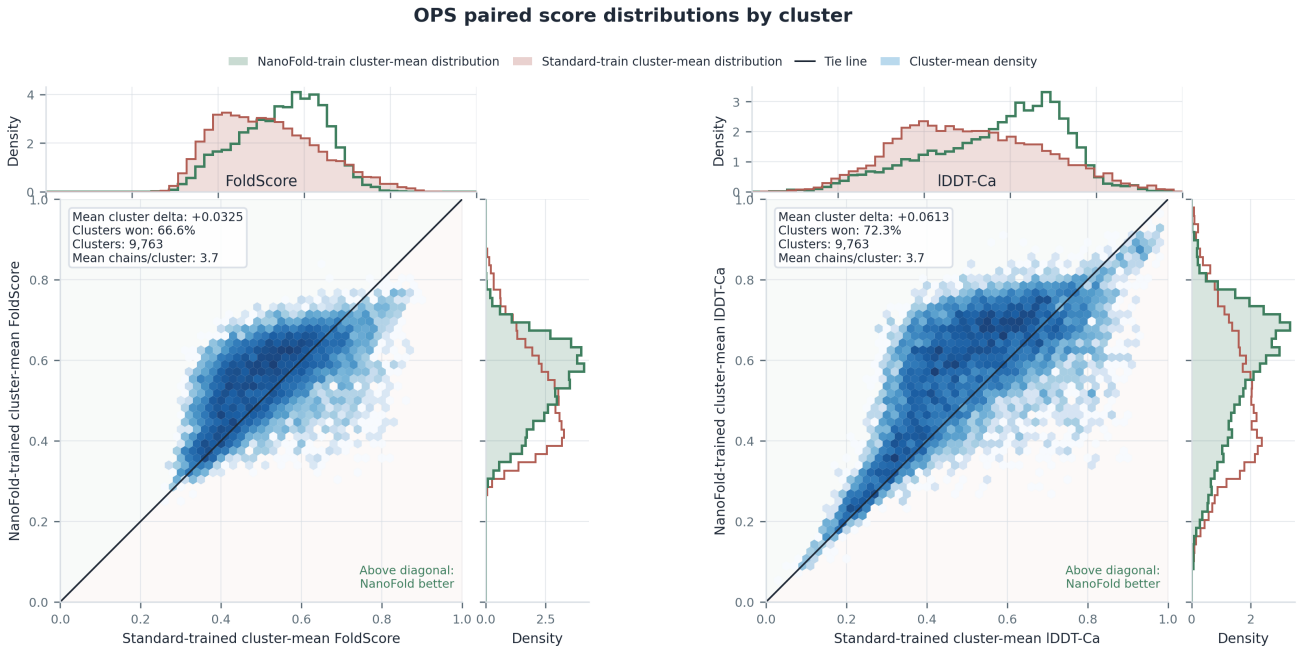


Figure 7. **OpenProteinSet generalization by sequence cluster.** Paired OpenProteinSet MMseqs2 cluster means compare standard-trained and NanoFold-trained AF2-medium models. Points above the diagonal favor NanoFold; cluster aggregation reduces family-size bias.

7. Conclusion

NanoFold starts from a simple premise: when experimentally resolved structures cannot be scaled on demand, one path to progress is to make the search for better inductive biases cheaper and more controlled. Its leakage-controlled, coverage-oriented splits and shared evaluation setup support matched comparisons of model-design choices that are too often entangled with changes to data and implementation.

Within this regime, NanoFold recovers the expected capacity ordering and recycling sensitivity and identifies recycling, loss, and curriculum changes that compose, raising AF2-medium public-validation FoldScore from 0.457 to 0.504. Longer-budget and broader OpenProteinSet checks show that the AF2-medium training signal extends beyond NanoFold's default horizon and validation pool to a disjoint OpenProteinSet-derived evaluation pool.

Together, these results position NanoFold as a hypothesis-screening instrument for data-constrained model development. Its fixed splits and common evaluation loop make model changes easier to isolate, compare, and retest. More broadly, NanoFold supports a practical research strategy when labels are fixed: screen broadly with controlled, compute-accessible experiments, then reserve sealed evaluation and production-scale compute for the highest-priority candidates.

Acknowledgments

We thank Lyra Labs for providing the compute to run these experiments. We also thank our anonymous reviewers for their constructive feedback, which improved the clarity and quality of the manuscript.

Software and Data

The training corpus, evaluation infrastructure, and reference codebase are publicly released to support the directions discussed in this paper.

Impact Statement

This paper advances reproducible machine learning benchmark design for protein structure prediction. Its principal societal consequences arise through improved scientific tooling for structural biology.

References

Abramson, J., Adler, J., Dunger, J., Evans, R., Green, T., Pritzel, A., Ronneberger, O., Willmore, L., Ballard, A. J., Bambrick, J., et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630 (8016):493–500, 2024. doi: 10.1038/s41586-024-07487-w.

Ahdritz, G., Bouatta, N., Kadyan, S., Jarosch, L., Berenberg, D., Fisk, I., Watkins, A. M., Ra, S., Bonneau, R.,

and AlQuraishi, M. OpenProteinSet: Training data for structural biology at scale. In *Advances in Neural Information Processing Systems 36*, pp. 4597–4609, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/0eb82171240776fe19da498bef3b1abe-Abstract-Datasets_and_Benchmarks.html.

Ahdritz, G., Bouatta, N., Floristean, C., Kadyan, S., Xia, Q., Gerecke, W., O'Donnell, T. J., Berenberg, D., Fisk, I., Zanichelli, N., et al. OpenFold: retraining AlphaFold2 yields new insights into its learning mechanisms and capacity for generalization. *Nature Methods*, 21(8):1514–1524, 2024. doi: 10.1038/s41592-024-02272-z.

AlQuraishi, M. ProteinNet: a standardized data set for machine learning of protein structure. *BMC Bioinformatics*, 20(1):311, 2019. doi: 10.1186/s12859-019-2932-0.

Baek, M., DiMaio, F., Anishchenko, I., Dauparas, J., Ovchinnikov, S., Lee, G. R., Wang, J., Cong, Q., Kinch, L. N., Schaeffer, R. D., et al. Accurate prediction of protein structures and interactions using a three-track neural network. *Science*, 373(6557):871–876, 2021. doi: 10.1126/science.abj8754.

Battiloro, C., Karaismailoğlu, E., Tec, M., Dasoulas, G., Audirac, M., and Dominici, F. E(n) equivariant topological neural networks. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/33b47b3d2441a17b95344cd635f3dd01-Abstract-Conference.html.

Blum, A. and Hardt, M. The ladder: A reliable leaderboard for machine learning competitions. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 1006–1014. PMLR, 2015. URL https://proceedings.mlr.press/v37/blum15.html.

Bodnar, C., Frasca, F., Wang, Y., Otter, N., Montufar, G. F., Lió, P., and Bronstein, M. Weisfeiler and lehman go topological: Message passing simplicial networks. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 1026–1037. PMLR, 2021. URL https://proceedings.mlr.press/v139/bodnar21a.html.

Burley, S. K., Bhikadiya, C., Bi, C., Bittrich, S., Chen, L., Crichlow, G. V., Christie, C. H., Dalenberg, K., Di Costanzo, L., Duarte, J. M., et al. RCSB protein data bank: powerful new tools for exploring 3d structures of biological macromolecules for basic and applied research and education in fundamental biology, biomedicine, biotechnology, bioengineering and energy sciences. *Nucleic Acids Research*, 49(D1):D437–D451, 2021. doi: 10.1093/nar/gkaa1038.

ByteDance AML AI4Science Team, Chen, X., Zhang, Y., Lu, C., Ma, W., Guan, J., Gong, C., Yang, J., Zhang, H., Zhang, K., Wu, S., Zhou, K., Yang, Y., Liu, Z., Wang, L., Shi, B., Shi, S., and Xiao, W. Protenix: Advancing structure prediction through a comprehensive AlphaFold3 reproduction. *bioRxiv*, 2025. doi: 10.1101/2025.01.08.631967. URL https://www.biorxiv.org/content/early/2025/01/11/2025.01.08.631967.

Candido, S., Hayes, T., Derry, A., Rao, R., Lin, Z., Verkuil, R., Wu, B. Z., Lee, J. S., Bruguera, E. S., Keval, J. A., Kopylov, M., Pak, J. E., Wu, W., Thomas, N., Mataraso, S., Hsu, A., Trotman-Grant, A. C., Fatras, K., dos Santos Costa, A., Badkundri, R., Akın, H., Oktay, D., Deaton, J., Montabana, E., Sitwala, H., Yu, Y., Wiggert, M., Carlin, D. A., Goering, A. W., Blazejewski, T., Sandora, M., Hla, M., Jia, T. Z., Kloker, L. H., Sofroniew, N. J., Uehara, M., Pannu, J., Bachas, S., Liu, D. S., Sercu, T., and Rives, A. Language modeling materializes a world model of protein biology. *bioRxiv*, 2026. doi: 10.64898/2026.06.03.729735. URL https://www.biorxiv.org/content/early/2026/06/04/2026.06.03.729735.

Chai Discovery, Boitreaud, J., Dent, J., McPartlon, M., Meier, J., Reis, V., Rogozhnikov, A., and Wu, K. Chai-1: Decoding the molecular interactions of life. *bioRxiv*, 2024. doi: 10.1101/2024.10.10.615955. URL https://www.biorxiv.org/content/early/2024/10/11/2024.10.10.615955.

Chandonia, J.-M., Guan, L., Lin, S., Yu, C., Fox, N. K., and Brenner, S. E. SCOPe: improvements to the structural classification of proteins–extended database to facilitate variant interpretation and machine learning. *Nucleic Acids Research*, 50(D1):D553–D559, 2022. doi: 10.1093/nar/gkab1054.

Chen, A., Dohan, D. M., and So, D. R. EvoPrompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems*, volume 36, pp. 7787–7817, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/184c1e18d00d7752805324da48ad25be-Abstract-Conference.html.

Chen, V. B., Arendall, W. B., Headd, J. J., Keedy, D. A., Immormino, R. M., Kapral, G. J., Murray, L. W., Richardson, J. S., and Richardson, D. C. MolProbity: all-atom structure validation for macromolecular crystallography. *Acta Crystallographica Section D: Biological Crystallography*, 66(1):12–21, 2010. doi: 10.1107/S0907444909042073.

Chen, Z., Chen, S., Ning, Y., Zhang, Q., Wang, B., Yu, B., Li, Y., Liao, Z., Wei, C., Lu, Z., Dey, V., Xue, M., Baker, F. N., Burns, B., Adu-Ampratwum, D., Huang, X., Ning, X., Gao, S., Su, Y., and Sun, H. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *The Thirteenth International Conference on Learning Representations*,

2025. URL https://openreview.net/forum?id=6z4YKr0GK6.

Cheng, H., Schaeffer, R. D., Liao, Y., Kinch, L. N., Pei, J., Shi, S., Kim, B.-H., and Grishin, N. V. ECOD: an evolutionary classification of protein domains. *PLoS Computational Biology*, 10(12):e1003926, 2014. doi: 10.1371/journal.pcbi.1003926.

Didi, K., Zhang, Z., Zhou, G., Reidenbach, D., Cao, Z., Cha, S., Geffner, T., Dallago, C., Tang, J., Bronstein, M. M., Steinegger, M., Kucukbenli, E., Vahdat, A., and Kreis, K. Scaling atomistic protein binder design with generative pretraining and test-time compute. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=qmCpJtFZra.

Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., and Roth, A. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015. doi: 10.1126/science.aaa9375.

Eijkelboom, F., Hesselink, R., and Bekkers, E. J. E(n) equivariant message passing simplicial networks. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 9071–9081. PMLR, 2023. URL https://proceedings.mlr.press/v202/eijkelboom23a.html.

Feng, Y., You, H., Zhang, Z., Ji, R., and Gao, Y. Hypergraph neural networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01):3558–3565, 2019. doi: 10.1609/aaai.v33i01.33013558.

Fu, L., Niu, B., Zhu, Z., Wu, S., and Li, W. CD-HIT: accelerated for clustering the next-generation sequencing data. *Bioinformatics*, 28(23):3150–3152, 2012. doi: 10.1093/bioinformatics/bts565.

Haas, J., Barbato, A., Behringer, D., Studer, G., Roth, S., Bertoni, M., Mostaguir, K., Gumienny, R., and Schwede, T. Continuous automated model evaluation (CAMEO) complementing the critical assessment of structure prediction in CASP12. *Proteins: Structure, Function, and Bioinformatics*, 86(S1):387–398, 2018. doi: 10.1002/prot.25431.

Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., and Leskovec, J. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems 33*, pp. 22118–22133, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/fb60d411a5c5b72b2e7d3527cfc84fd0-Abstract.html.

Huang, K., Fu, T., Gao, W., Zhao, Y., Roohani, Y. H., Leskovec, J., Coley, C. W., Xiao, C., Sun, J., and Zitnik, M. Therapeutics data commons: Machine learning datasets and tasks for drug discovery and development. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. URL https://openreview.net/forum?id=8nvgnORnoWr.

Huang, P.-S., Boyken, S. E., and Baker, D. The coming of age of de novo protein design. *Nature*, 537(7620):320–327, 2016. doi: 10.1038/nature19946.

Huang, Q., Vora, J., Liang, P., and Leskovec, J. MLAgentBench: Evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 20271–20309. PMLR, 2024. URL https://proceedings.mlr.press/v235/huang24y.html.

Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

Kabsch, W. A solution for the best rotation to relate two sets of vectors. *Acta Crystallographica Section A*, 32(5):922–923, 1976. doi: 10.1107/S0567739476001873.

Kryshtafovych, A., Schwede, T., Topf, M., Fidelis, K., and Moult, J. Critical assessment of methods of protein structure prediction (CASP)–round XV. *Proteins: Structure, Function, and Bioinformatics*, 91(12):1539–1549, 2023. doi: 10.1002/prot.26617.

Kryshtafovych, A., Schwede, T., Topf, M., Fidelis, K., and Moult, J. Progress and bottlenecks for deep learning in computational structure biology: CASP round XVI. *Proteins: Structure, Function, and Bioinformatics*, 94(1):5–14, 2026. doi: 10.1002/prot.70076.

La, H., Gupta, A., Morehead, A., Cheng, J., and Zhang, M. MegaFold: Efficient training of next-generation 3d attention protein models on cross-platform GPUs. *arXiv preprint arXiv:2506.20686*, 2025. doi: 10.48550/arXiv.2506.20686. URL https://arxiv.org/abs/2506.20686. Version 2, revised 13 June 2026.

Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., Smetanin, N., Verkuil, R., Kabeli, O., Shmueli, Y., et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.

Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., and Ha, D. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. doi: 10.48550/arXiv.2408.06292. URL https://arxiv.org/abs/2408.06292.

Ma, W., Liu, Z., Yang, J., Lu, C., Zhang, H., and Xiao, W. From dataset curation to unified evaluation: Revisiting structure prediction benchmarks with PXMeter. *bioRxiv*, 2025. doi: 10.1101/2025.07.17.664878. URL https://www.biorxiv.org/content/early/2025/07/22/2025.07.17.664878.

Mariani, V., Biasini, M., Barbato, A., and Schwede, T. lDDT: a local superposition-free score for comparing

protein structures and models using distance difference tests. *Bioinformatics*, 29(21):2722–2728, 2013. doi: 10.1093/bioinformatics/btt473.

McInnes, L., Healy, J., and Melville, J. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. doi: 10.48550/arXiv.1802.03426. URL https://arxiv.org/abs/1802.03426.

Olechnovic, K., Kulberkyte, E., and Venclovas, C. CAD-score: a new contact area difference-based function for evaluation of protein structural models. *Proteins: Structure, Function, and Bioinformatics*, 81(1):149–162, 2013. doi: 10.1002/prot.24172.

Passaro, S., Corso, G., Wohlwend, J., Reveiz, M., Thaler, S., Somnath, V. R., Getz, N., Portnoi, T., Roy, J., Stark, H., Kwabi-Addo, D., Beaini, D., Jaakkola, T., and Barzilay, R. Boltz-2: Towards accurate and efficient binding affinity prediction. *bioRxiv*, 2025. doi: 10.1101/2025.06.14.659707. URL https://doi.org/10.1101/2025.06.14.659707.

Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J. R., Ellenberg, J. S., Wang, P., Fawzi, O., Kohli, P., and Fawzi, A. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.

Sadybekov, A. V. and Katritch, V. Computational approaches streamlining drug discovery. *Nature*, 616(7958): 673–685, 2023. doi: 10.1038/s41586-023-05905-z.

Sillitoe, I., Bordin, N., Dawson, N., Waman, V. P., Ashford, P., Scholes, H. M., Pang, C. S. M., Woodridge, L., Rauer, C., Sen, N., Abbasian, M., Le Cornu, S., Lam, S. D., Berka, K., Varekova, I. H., Svobodova, R., Lees, J., and Orengo, C. A. CATH: increased structural coverage of functional space. *Nucleic Acids Research*, 49(D1):D266–D273, 2021. doi: 10.1093/nar/gkaa1079.

Simpkin, A. J., Mesdaghi, S., Sánchez Rodríguez, F., Elliott, L., Murphy, D. L., Kryshtafovych, A., Keegan, R. M., and Rigden, D. J. Tertiary structure assessment at CASP15. *Proteins: Structure, Function, and Bioinformatics*, 91 (12):1616–1635, 2023. doi: 10.1002/prot.26593.

Sorscher, B., Geirhos, R., Shekhar, S., Ganguli, S., and Morcos, A. S. Beyond neural scaling laws: beating power law scaling via data pruning. In *Advances in Neural Information Processing Systems 35*, pp. 19523–19536, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/7b75da9b61eda40fa35453ee5d077df6-Abstract-Conference.html.

Steinegger, M. and Söding, J. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35(11):1026–1028, 2017. doi: 10.1038/nbt.3988.

Suzek, B. E., Wang, Y., Huang, H., McGarvey, P. B., Wu, C. H., and UniProt Consortium. UniRef clusters: a comprehensive and scalable alternative for improving sequence similarity searches. *Bioinformatics*, 31(6):926–932, 2015. doi: 10.1093/bioinformatics/btu739.

The OpenFold3 Team. OpenFold3-preview2 technical report. https://portal.openfold.omsf.io/reports/of3p2_technical_report.pdf, 2026. Research preview technical report.

Varadi, M., Anyango, S., Deshpande, M., Nair, S., Natassia, C., Yordanova, G., Yuan, D., Stroe, O., Wood, G., Laydon, A., et al. AlphaFold protein structure database: massively expanding the structural coverage of protein-sequence space with high-accuracy models. *Nucleic Acids Research*, 50(D1):D439–D444, 2022. doi: 10.1093/nar/gkab1061.

Watson, J. L., Juergens, D., Bennett, N. R., Trippe, B. L., Yim, J., Eisenach, H. E., Ahern, W., Borst, A. J., Ragotte, R. J., Milles, L. F., et al. De novo design of protein structure and function with RFdiffusion. *Nature*, 620 (7976):1089–1100, 2023. doi: 10.1038/s41586-023-06415-8.

Westbrook, J. D., Young, J. Y., Shao, C., Feng, Z., Guranovic, V., Lawson, C. L., Vallat, B., Adams, P. D., Berrisford, J. M., Bricogne, G., et al. PDBx/mmCIF ecosystem: Foundational semantic tools for structural biology. *Journal of Molecular Biology*, 434(11):167599, 2022. doi: 10.1016/j.jmb.2022.167599.

Wohlwend, J., Corso, G., Passaro, S., Getz, N., Reveiz, M., Leidal, K., Swiderski, W., Atkinson, L., Portnoi, T., Chinn, I., Silterra, J., Jaakkola, T., and Barzilay, R. Boltz-1: Democratizing biomolecular interaction modeling. *bioRxiv*, 2024. doi: 10.1101/2024.11.19.624167. URL https://doi.org/10.1101/2024.11.19.624167.

Wu, Z., Ramsundar, B., Feinberg, E. N., Gomes, J., Geniesse, C., Pappu, A. S., Leswing, K., and Pande, V. MoleculeNet: a benchmark for molecular machine learning. *Chemical Science*, 9(2):513–530, 2018. doi: 10.1039/C7SC02664A.

wwPDB Consortium. Protein data bank: the single global archive for 3d macromolecular structure data. *Nucleic Acids Research*, 47(D1):D520–D528, 2019. doi: 10.1093/nar/gky949.

Yamada, Y., Lange, R. T., Lu, C., Hu, S., Lu, C., Foerster, J., Clune, J., and Ha, D. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025. doi: 10.48550/arXiv.2504.08066. URL https://arxiv.org/abs/2504.08066.

Zemla, A. LGA: a method for finding 3d similarities in protein structures. *Nucleic Acids Research*, 31(13):3370–3374, 2003. doi: 10.1093/nar/gkg571.

Zhu, F., Nowaczynski, A., Li, R., Xin, J., Song, Y., Marcinkiewicz, M., Eryilmaz, S. B., Yang, J., and An-

dersch, M. ScaleFold: Reducing AlphaFold initial training time to 10 hours. *arXiv preprint arXiv:2404.11068*, 2024. doi: 10.48550/arXiv.2404.11068. URL https://arxiv.org/abs/2404.11068.

A. Technical appendices and supplementary material

A.1. Supplementary dataset details

A.1.1. Diagnostic formulas

This appendix gives the exact diagnostics used in Section 3.3. For a categorical metadata field with bucket set $\mathcal{B}$, let $p_S(b)$ be the empirical fraction of chains in split S that fall in bucket $b \in \mathcal{B}$, and let $p_R(b)$ be the corresponding reference distribution, either the eligible source pool or another split. We report Jensen–Shannon divergence,

$$\mathrm{JSD}(p_S, p_R) = \frac{1}{2}\mathrm{KL}(p_S \,\|\, m) + \frac{1}{2}\mathrm{KL}(p_R \,\|\, m), \qquad m = \frac{1}{2}(p_S + p_R),$$

with zero-probability terms omitted in the usual way. We also report total variation distance,

$$\mathrm{TV}(p_S, p_R) = \frac{1}{2}\sum_{b\in\mathcal{B}} |p_S(b) - p_R(b)|,$$

and maximum bin deviation,

$$\Delta_{\max}(p_S, p_R) = \max_{b\in\mathcal{B}} |p_S(b) - p_R(b)|.$$

The embedding-space diagnostics use cosine distance in the 2,560-dimensional ESMC 6B mean-embedding space. For chain x, the nearest-train distance is

$$r_T(x) = \min_{t\in T} d_{\cos}(\phi(x), \phi(t)),$$

where T is the public-training split and $\phi(x)$ is the mean-pooled ESMC embedding. To normalize for local density, let $\rho_k(x)$ be the distance from x to its kth nearest neighbor in the eligible source pool, excluding x itself. The density-normalized gap is

$$g_{T,k}(x) = \frac{r_T(x)}{\rho_k(x)}.$$

Coverage at a local radius is reported as

$$C_k(S;\alpha) = \frac{1}{|S|}\sum_{x\in S} \mathbf{1}\{r_T(x) \le \alpha\rho_k(x)\},$$

with $\alpha \in \{1, 2\}$ in the reported tables. Coarse-region coverage clusters ESMC embeddings into K regions and reports both the fraction of clusters touched by a split and the fraction of source-pool mass contained in touched clusters.

A.1.2. Conditional randomization protocol

The randomization analysis compares the committed public split against alternative valid splits drawn under the same constraints. Each randomized split preserves the official candidate universe, chain eligibility filters, unit-level assignment, PDB-entry disjointness, MMseqs2 cluster disjointness, target training and public-validation sizes, a reserve equal in size to the hidden-validation set, and the same stratification fields used by the committed allocator. For a statistic Q, the reported empirical percentile is

$$\widehat{P}_{\le}(Q_{\mathrm{obs}}) = \frac{1}{B}\sum_{b=1}^{B} \mathbf{1}\{Q_b \le Q_{\mathrm{obs}}\}, \qquad B = 1{,}000.$$

This conditional randomization check evaluates the committed split against valid alternatives generated under the same construction rules. Its design-specific null distribution measures whether the committed split has typical difficulty and balance relative to those alternatives.

Conditional randomization. Finally, the committed NanoFold split is compared against alternative valid splits drawn under the same official constraints. We generate $B = 1{,}000$ alternative public-training and public-validation splits respecting the same candidate universe, sequence-cluster disjointness, PDB-entry disjointness, training and validation sizes, a reserve equal in size to the hidden-validation set, and stratification fields. For a diagnostic statistic Q, the empirical percentile $\widehat{P}_{\le} = \frac{1}{B}\sum_b \mathbf{1}\{Q_b \le Q_{\mathrm{obs}}\}$ reports where the committed split falls relative to valid randomized alternatives. Table 5 reports the comparison. Embedding-space diagnostics lie inside the central randomized range; public max JSD is above the

Table 3. **Nearest-train coverage in ESMC embedding space.** Cosine distances in the 2,560-dimensional ESMC 6B mean-embedding space, with density-normalized gap $g_{T,50}(x) = r_T(x)/\rho_{50}(x)$.

	Nearest-train distance			Density-normalized gap ($k = 50$)		
Set	p50	p90	p95	p50	Within ρ_{50}	Within $2\rho_{50}$
Non-train pool	0.0980	0.1821	0.2549	0.894	0.617	0.794
Public validation	0.0847	0.1274	0.1435	0.772	0.990	0.998
Hidden validation	0.0843	0.1300	0.1471	0.774	0.990	0.999

Table 4. **Coarse ESMC cluster occupancy.** Fraction of clusters touched by each split, and source-pool mass covered, at $K = 100$ and $K = 200$ regions.

	$K = 100$		$K = 200$	
Split	Clusters	Mass	Clusters	Mass
Train	0.860	0.874	0.835	0.837
Public validation	0.760	0.804	0.695	0.746
Hidden validation	0.770	0.806	0.685	0.735

randomized p95 but remains 0.703×10^{-3}, far below the official balance gate. In ESMC space, public validation is slightly closer to training than the randomized median, with nearest-train p50 of 0.0847 versus randomized 0.0849, p95 of 0.1436 versus 0.1443, and public-validation coverage at the global median ρ_{50} of 0.7352 versus 0.7281; broader non-train coverage and touched-cluster mass remain typical.

Table 5. **Conditional randomization diagnostics.** Comparison of the committed public split with 1,000 alternative valid splits sampled under the same official constraints. Lower is better for JSD and nearest-train distance; higher is better for coverage measures. Coverage rows use the global median 50-neighbor radius for each randomized split.

Metric	Observed	Random p05	Random p50	Random p95
Public max JSD ($\times 10^{-3}$)	0.703	0.423	0.423	0.423
Public val. nearest-train p50	0.0847	0.0834	0.0849	0.0865
Public val. nearest-train p95	0.1436	0.1403	0.1443	0.1485
Public val. global-ρ_{50} coverage	0.7352	0.7067	0.7281	0.7480
Non-train global-ρ_{50} coverage	0.6774	0.6740	0.6791	0.6843
$K = 100$ touched-cluster mass	0.8741	0.8637	0.8808	0.8808

A.1.3. STANDARD DISJOINT-CLUSTER BASELINE

The whole-cluster baseline in Figure 2 is a deliberately simpler alternative to the official allocator. It samples disjoint MMseqs2/PDB-connected units into training and validation splits without the official density-aware and structural balancing procedure. This baseline satisfies the same high-level leakage constraint but does not actively control how a limited budget is distributed across splits in the eligible protein universe. The figure overlays both procedures on the same UMAP coordinates and fixed-grid hex bins. Gray points show the broader OpenProteinSet/PDB universe, while colored hexes show unique records per UMAP bin for the selected split. The comparison demonstrates that disjointness alone is insufficient: a standard whole-cluster procedure can satisfy leakage rules while leaving visibly different density and coverage patterns.

A.2. Paper experiment protocol

A.2.1. SCOPE OF THE REPORTED EXPERIMENTS

All experiments reported here use the public-training and public-validation splits. Sealed hidden labels remain reserved for official leaderboard ranking, so the core learning curves characterize development-time training signals. OpenAI's GPT-5.5, running in the Codex harness, configured and operated the calibration and ablation studies by creating model configurations, provisioning training infrastructure, launching runs, and monitoring outputs within the author-specified questions and fixed

NanoFold protocol.

The core manuscript figures use the following analysis panels:

- **Scaling:** completed tiny, medium, and large minAlphaFold2 runs under the corrected AF2-style default training recipe.
- **FAPE policy:** medium-model backbone-FAPE clamp/unclamp policy ablations.
- **Recycling policy:** medium-model recycling-depth and fixed-vs-sampled recycling ablations.
- **Representative ablations:** a compact cross-ablation panel containing the medium default, fully unclamped FAPE, no fine-tune loss, fixed-four recycling, and the best combined setting.

Other exploratory and diagnostic runs are excluded from the core manuscript figures described here.

A.2.2. Common data and training setup

Table 6 summarizes the shared configuration. All stochastic runs use seed 0.

Table 6. **Shared training configuration for core public-validation experiments.**

Setting	Value
Training chains	10,000 public-training chains
Validation chains	1,000 public-validation chains
Hidden validation	Reserved for official leaderboard evaluation
Crop length	256 residues
Training crop	Random contiguous crop
Validation crop	Center crop
MSA rows	192 primary MSA rows, 64 extra MSA rows
MSA sampling	Random rows during training, top rows during validation
Microbatch size	1 chain
Gradient accumulation	8 microbatches
Effective batch size	8 chains per optimizer step
Optimizer	Adam
Learning rate	10^{-3}
Adam coefficients	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-6}$
Weight decay	0
Gradient clipping	Global norm clipped to 0.1
Mixed precision	Disabled
Training length	30,000 optimizer steps, or 240,000 samples
Evaluation cadence	Every 3,000 optimizer steps
Warmup	333 optimizer steps
Learning-rate decay	Factor 0.95 at step 16,695
Default fine-tune phase	Starts at step 26,088; ramps over 2,250 steps
Default fine-tune learning-rate scale	0.5 after the fine-tune boundary

All core training runs use the same NanoFold public-training and public-validation splits:

$$|T| = 10{,}000, \qquad |V_{\mathrm{pub}}| = 1{,}000.$$

The sealed hidden-validation split is reserved for official evaluation. Inputs are cropped to 256 residues, with random crops during training and center crops during validation. Training MSA rows are sampled randomly; validation uses the top MSA rows. Unless explicitly ablated, runs use an MSA depth of 192, an extra-MSA depth of 64, batch size 1, gradient accumulation over 8 microbatches, Adam with learning rate 10^{-3}, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-6}$, no weight decay, global gradient clipping at 0.1, and no mixed precision.

Every core run is trained for 30,000 optimizer steps. With batch size 1 and 8-step gradient accumulation, this corresponds to

$$30{,}000 \times 8 = 240{,}000$$

training samples. We evaluate on the public-validation split every 3,000 optimizer steps and save checkpoints at the same interval. The checkpoint set used for curves is

$$0, 3{,}000, 6{,}000, 9{,}000, 12{,}000, 15{,}000, 18{,}000, 21{,}000, 24{,}000, 27{,}000, 30{,}000.$$

All curves in the core paper are therefore directly comparable in optimizer-step and sample-budget units.

A.2.3. Model profiles

The experiments use minAlphaFold2 profiles derived from AlphaFold-style components but reduced to make repeated small-data training feasible. Table 7 summarizes the profiles that appear in the core figures. The medium profile is the default ablation size. The tiny and large profiles appear in the scaling analyses, and the large profile is also used in the public-validation structure-example and loss-component figures in the appendix.

Table 7. **minAlphaFold2 profiles used in core figures.** Parameter counts are measured by the NanoFold runner.

Profile	Parameters	Evoformer blocks	Default max recycles
tiny	88,898	1	1
medium	3,106,642	4	2
large	94,159,450	48	4

The medium profile uses MSA-, single-, and pair-channel widths of 128, 192, and 64, respectively, with four Evoformer blocks, a four-layer structure module, and eight IPA heads. The tiny profile uses channel widths of 32, 32, and 16, one Evoformer block, a two-layer structure module, and four IPA heads. The large profile uses channel widths of 256, 384, and 128, 48 Evoformer blocks, an eight-layer structure module, and 12 IPA heads; its default scaling run samples 1–4 training recycles and evaluates with four fixed recycles.

A.2.4. Learning-rate and fine-tuning schedule

The schedule follows the AlphaFold two-stage recipe (Jumper et al., 2021; Ahdritz et al., 2024) scaled to the 30,000-step paper budget. For the default schedule, the initial objective is used until step 26,088. Fine-tuning begins at step 26,088 and ramps over 2,250 steps, becoming fully active at step 28,338. The learning rate warms up for 333 steps, decays by a factor of 0.95 at step 16,695, and is multiplied by 0.5 after the fine-tuning boundary. The no-fine-tune ablation sets the fine-tune start to the final step and keeps the initial loss active throughout all optimizer updates.

A.2.5. Training loss

The initial loss is NanoFold's AlphaFold-inspired supervised objective, adapted from AlphaFold2 and OpenFold (Jumper et al., 2021; Ahdritz et al., 2024). The displayed weights describe NanoFold's implementation rather than a term-by-term transcription of the cited objectives; in particular, NanoFold logs and weights intermediate backbone-FAPE and torsion terms separately.

$$\begin{aligned}\mathcal{L}_{\text{init}} = 0.5\,\mathcal{L}_{\text{aux-bbFAPE}} + 0.5\,\mathcal{L}_{\text{allatomFAPE}} + \mathcal{L}_{\text{torsion}} \\ + 0.3\,\mathcal{L}_{\text{distogram}} + 2.0\,\mathcal{L}_{\text{maskedMSA}} + 0.01\,\mathcal{L}_{\text{pLDDT}}.\end{aligned}$$

Here $\mathcal{L}_{\text{aux-bbFAPE}}$ is the per-structure-module-iteration backbone FAPE, $\mathcal{L}_{\text{allatomFAPE}}$ is the final all-atom FAPE term, $\mathcal{L}_{\text{torsion}}$ includes the torsion-angle and angle-normalization terms, $\mathcal{L}_{\text{distogram}}$ is the Cβ–Cβ distance-bin cross-entropy loss, $\mathcal{L}_{\text{maskedMSA}}$ is the BERT-style MSA reconstruction loss, and $\mathcal{L}_{\text{pLDDT}}$ is the pLDDT confidence-head loss.

After the fine-tuning ramp is fully active, the fine-tuning objective is

$$\mathcal{L}_{\text{ft}} = \mathcal{L}_{\text{init}} + 1.0\,\mathcal{L}_{\text{viol}} + 0.01\,\mathcal{L}_{\text{exp}} + 0.1\,\mathcal{L}_{\text{PAE}},$$

where $\mathcal{L}_{\text{viol}}$ is the structural violation loss, $\mathcal{L}_{\text{exp}}$ is the experimentally resolved atom loss, and $\mathcal{L}_{\text{PAE}}$ is the predicted aligned error (PAE)/TM-head loss. During the ramp interval, the runner blends the initial and fine-tuning losses and logs the ramp weight. All component losses, weighted component losses, training loss, validation loss, public-validation lDDT-Cα, RMSD-Cα, and FoldScore components are saved when available.

A.2.6. FAPE CLAMP/UNCLAMP POLICIES

Let $\mathcal{L}_{\text{bbFAPE}}^{\text{clamp}}$ denote backbone FAPE with the 10 Å distance clamp, and let $\mathcal{L}_{\text{bbFAPE}}^{\text{unclamp}}$ denote the unclamped backbone FAPE. The implementation writes the active clamp weight as w and evaluates

$$w\,\mathcal{L}_{\text{bbFAPE}}^{\text{clamp}} + (1-w)\,\mathcal{L}_{\text{bbFAPE}}^{\text{unclamp}}.$$

Side-chain/all-atom FAPE remains clamped. The core paper compares the following policies:

- **AF2 default / batchwise 90/10:** training samples a scalar $w \in \{0, 1\}$ per optimizer step with $\Pr(w = 1) = 0.9$; validation uses the deterministic expectation $w = 0.9$.
- **Soft mix 90/10:** training and validation always use $w = 0.9$, directly optimizing the expectation of the clamped/unclamped mixture.
- **Samplewise 90/10 and 50/50:** training samples w independently per example with a clamping probability of 0.9 or 0.5; validation uses the corresponding deterministic expectation.
- **Fully unclamped:** training and validation use $w = 0$.
- **Unclamp at fine-tune:** training is fully clamped before the fine-tuning boundary and fully unclamped after the fine-tuning boundary.

A.2.7. RECYCLING POLICIES

The AF2-style default samples the number of training recycles uniformly from $1, \ldots, N_{\max}$ and evaluates deterministically at $N_{\max}$. Thus the medium default samples one or two recycles during training and evaluates with two recycles. The recycling sweep sets $N_{\max} \in \{2, 4, 6, 8\}$ with the same sampled training rule and fixed-maximum evaluation rule. The explicit fixed-four ablation trains and evaluates with exactly four recycles, without train-time recycle sampling.

A.2.8. CORE RUN DEFINITIONS

Table 8 gives the full experimental definition for every run used in the core public-validation learning-curve figures. "Sampled" recycling means that the number of training recycles is sampled uniformly from $1, \ldots, N_{\max}$, while validation uses $N_{\max}$. "Fixed" means the same recycle count is used during both training and validation. Unless noted otherwise, the fine-tune schedule is the default two-stage schedule in Appendix A.2.4.

Table 8. **Run definitions for the core ablation figures.**

Run	Profile	Training recycles	Validation recycles	Backbone FAPE / loss schedule
tiny AF2 default	tiny	sampled 1	1	batchwise 90/10; default fine-tune
medium AF2 default	medium	sampled 1–2	2	batchwise 90/10; default fine-tune
large AF2 default	large	sampled 1–4	4	batchwise 90/10; default fine-tune
FAPE batchwise 90/10	medium	sampled 1–2	2	batchwise 90/10; default fine-tune
FAPE soft mix 90/10	medium	sampled 1–2	2	deterministic 90/10 mix; default fine-tune
FAPE samplewise 90/10	medium	sampled 1–2	2	samplewise 90/10; default fine-tune
FAPE samplewise 50/50	medium	sampled 1–2	2	samplewise 50/50; default fine-tune
FAPE fully unclamped	medium	sampled 1–2	2	fully unclamped; default fine-tune
FAPE unclamp at fine-tune	medium	sampled 1–2	2	clamped before fine-tune, unclamped after
one cycle	medium	fixed 1	1	batchwise 90/10; default fine-tune
sampled 1–4 recycles	medium	sampled 1–4	4	batchwise 90/10; default fine-tune
sampled 1–6 recycles	medium	sampled 1–6	6	batchwise 90/10; default fine-tune
sampled 1–8 recycles	medium	sampled 1–8	8	batchwise 90/10; default fine-tune
fixed 4 recycles	medium	fixed 4	4	batchwise 90/10; default fine-tune
no fine-tune loss	medium	sampled 1–2	2	batchwise 90/10; initial loss for all steps
fixed 4 + unclamped + no fine-tune	medium	fixed 4	4	fully unclamped; initial loss for all steps

A.3. Core experimental results

Table 9 lists the runs used in the core public-validation learning-curve figures, including the large scaling run. The final public-validation lDDT-Cα and RMSD-Cα values are reported from the final checkpoint used in the manuscript curves.

Table 9. **Core paper public-validation runs.** All runs use 30,000 optimizer steps and 240,000 training samples. Wall time is measured by the RunPod H100 runner.

Run	Main difference from medium default	lDDT-Cα	RMSD-Cα	H100 h
tiny AF2 default	tiny profile, maximum recycle count 1	0.235	25.35	10.02
medium AF2 default	medium profile, sampled 1–2 recycles, batchwise FAPE	0.293	19.56	15.75
large AF2 default	large profile, sampled 1–4 recycles, batchwise FAPE	0.658	7.51	137.25
FAPE batchwise 90/10	AF2-default comparator	0.293	19.56	16.55
FAPE soft mix 90/10	deterministic 90/10 FAPE mixture	0.286	18.13	16.43
FAPE samplewise 90/10	per-example stochastic 90/10 FAPE	0.305	18.36	17.97
FAPE samplewise 50/50	per-example stochastic 50/50 FAPE	0.385	12.71	16.84
FAPE fully unclamped	unclamped backbone FAPE throughout	0.450	10.54	15.72
FAPE unclamp at fine-tune	unclamp only after fine-tune boundary	0.319	15.85	16.63
one cycle	no iterative recycling	0.270	20.28	10.09
sampled 1–4 recycles	maximum/evaluation recycles: 4	0.291	20.03	19.53
sampled 1–6 recycles	maximum/evaluation recycles: 6	0.302	19.10	22.04
sampled 1–8 recycles	maximum/evaluation recycles: 8	0.313	16.39	23.47
fixed 4 recycles	four recycles in training/evaluation	0.324	14.54	19.28
no fine-tune loss	initial objective for all 30,000 steps	0.413	12.66	17.90
fixed 4 + unclamped + no fine-tune	fixed 4 recycles, fully unclamped FAPE, no fine-tune	0.534	7.74	23.38

Interpretation of the core runs. The scale panel shows that NanoFold remains unsaturated: medium improves over tiny, and large improves further, under the same data and sample budget. The FAPE panel isolates the largest objective-level signal in these experiments. Fully unclamped backbone FAPE substantially improves both lDDT-Cα and RMSD-Cα, while partial unclamping gives intermediate behavior. The recycling panel shows a modest but coherent benefit from using more recycles, with fixed-four recycling outperforming sampled-four under this small-data setup. The representative panel combines these observations and shows that the strongest observed medium-profile setting uses fixed-four recycling, fully unclamped backbone FAPE, and no fine-tuning loss phase.

A.4. Supplementary lDDT and loss-component trajectories

The main text reports FoldScore curves because FoldScore is the benchmark-level aggregate metric. For transparency, this appendix also reports the corresponding public-validation lDDT-Cα curves and the per-run training loss-component trajectories for the ablations discussed in Section 5. The lDDT-Cα panels use the same checkpoint set as the FoldScore panels. The loss-component panels show the logged supervised objective terms for each run, which helps distinguish early optimization effects from changes that appear near the fine-tuning boundary or final checkpoint. Public-validation lDDT-Cα curves are shown in Figures 8, 9, and 10, while the corresponding loss-component trajectories are shown in Figures 11, 12, 13, and 14.

We first compare the public-validation lDDT-Cα trajectories for model scale and recycling depth. These curves are the lDDT-Cα counterparts of the main-text FoldScore comparisons under the same checkpoints and sample budget.

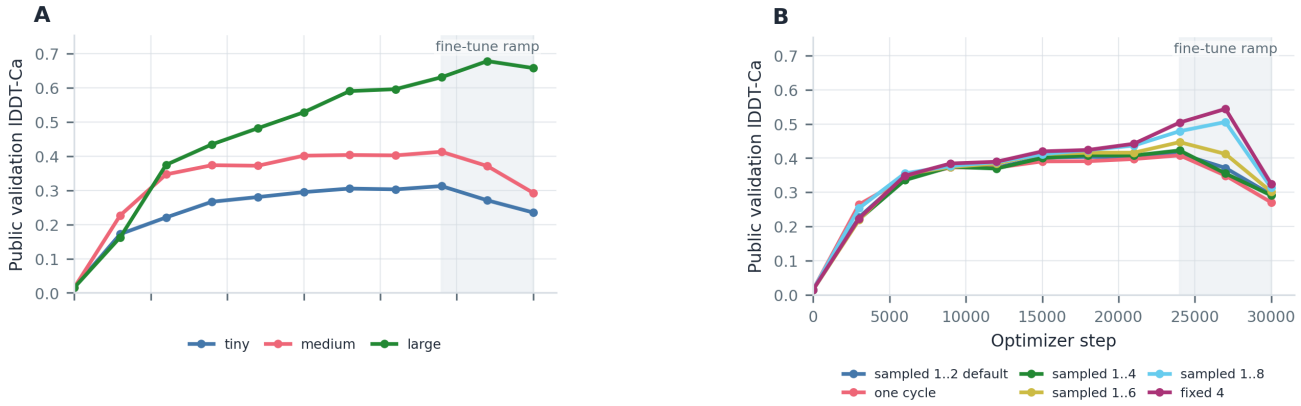
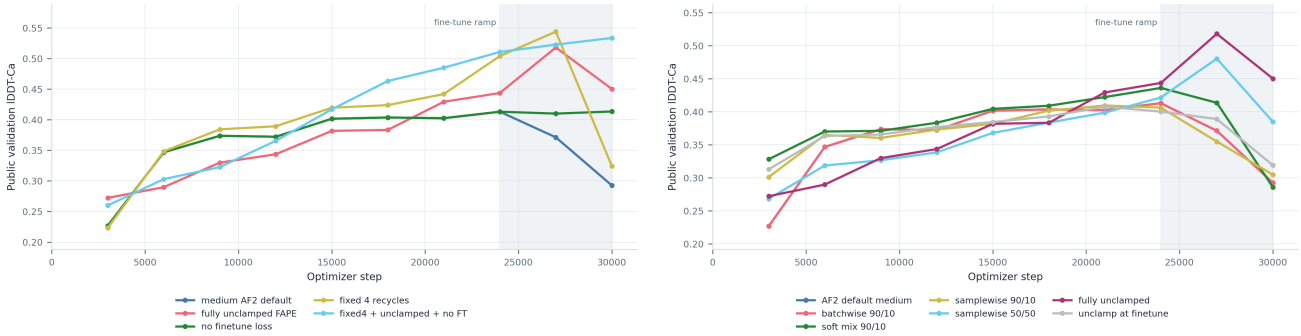


Figure 8. **Supplementary public-validation lDDT-Cα curves for scaling and recycling.** These are the lDDT-Cα counterparts to the FoldScore scaling and recycling curves in the main text: Panel A shows model scaling, and Panel B shows recycling-policy ablations. Both panels use the same optimizer-step x-axis, whose numeric labels are shown in Panel B. The shaded span marks the fine-tuning phase.

The objective-focused view separates the effect of backbone-FAPE clamping from the combined recipe. The no-fine-tune run highlights the complementarity of the metrics: it improves lDDT-Cα while lowering the composite FoldScore.



(a) Representative ablations

(b) Backbone-FAPE clamping policies

Figure 9. **Supplementary public-validation lDDT-Cα curves for objective ablations.** Fully unclamped backbone FAPE and the combined fixed-four/unclamped/no-fine-tune setting produce the strongest lDDT-Cα gains, consistent with their ordering by FoldScore. Removing the fine-tuning loss phase raises lDDT-Cα (0.413 versus 0.293 for the medium default) while lowering FoldScore (0.400 versus 0.457), reflecting the steric-clash and backbone terms included in FoldScore. Shaded spans mark the fine-tuning phase.

The dedicated recycling-policy view resolves the individual settings summarized together in Figure 8. It holds the medium profile and all non-recycling choices fixed, so the curves isolate effective depth under a common data, optimizer, and loss schedule. Figure 10 shows the fixed-four setting leading late in training, followed by sampled 1–8 recycling.

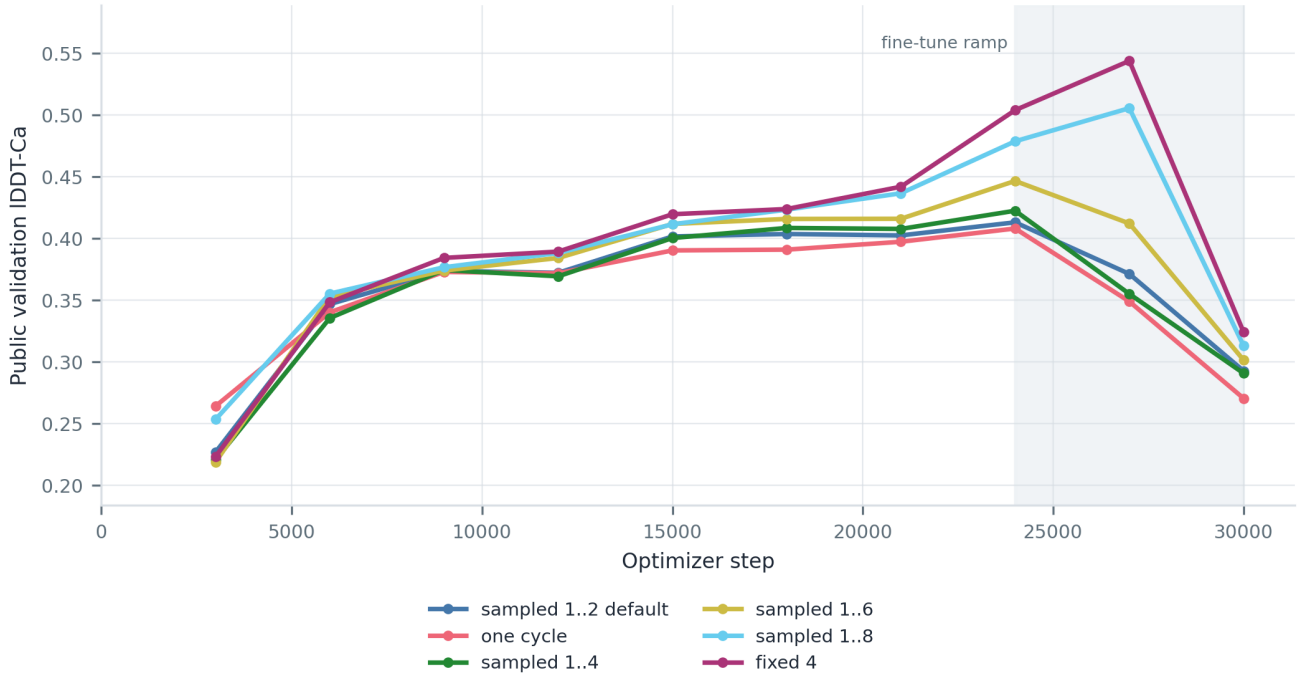
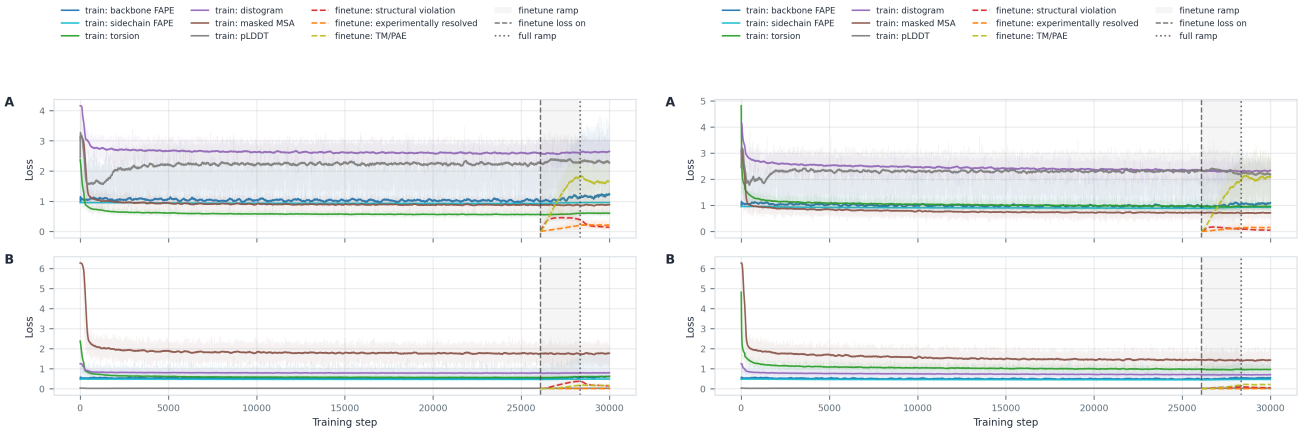


Figure 10. **Supplementary public-validation lDDT-Cα curves for recycling policy.** Recycling depth affects lDDT-Cα in the same direction as FoldScore, with the fixed-four recycling setting outperforming sampled-four recycling under the 240,000-sample training budget. The shaded span marks the fine-tuning phase.

Training-loss trajectories. We next report the logged training components underlying the validation curves. The scale comparison establishes how the shared AF2-style objective behaves as model capacity increases.

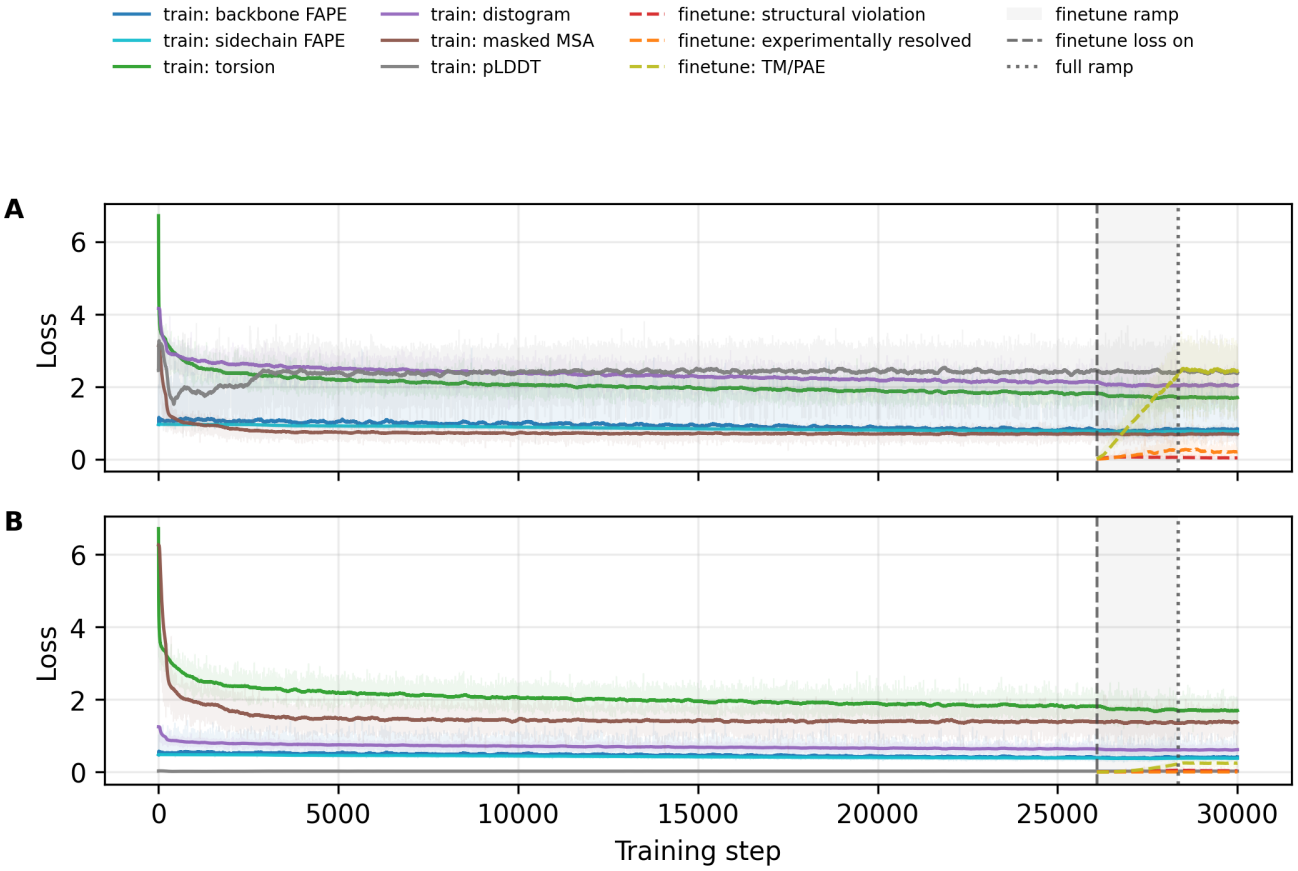


(a) Tiny AF2 default

(b) Medium AF2 default

Figure 11. **Loss-component trajectories for the scaling runs.** In each subfigure, Panel A shows raw loss components and Panel B shows weighted contributions. The tiny and medium profiles use the same AF2-style default recipe under the same fixed data budget.

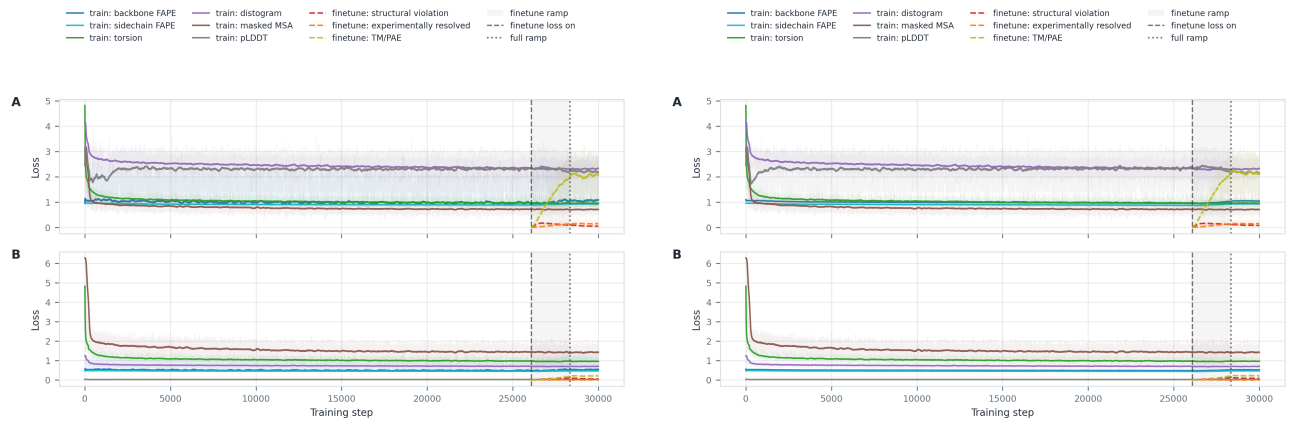
Across both profiles, the logged objective terms stabilize rapidly after the initial optimization transient. The large-profile trajectory is shown at full width to keep its densely overlaid raw and weighted components legible.



(c) Large AF2 default

Figure 11. **Loss-component trajectories for the scaling runs (continued).** The large profile uses the same AF2-style default recipe and achieves the strongest validation structure scores under the same fixed data budget.

The backbone-FAPE sweep then isolates how clamping policy changes the supervised objective terms while leaving the rest of the medium-model setup fixed.

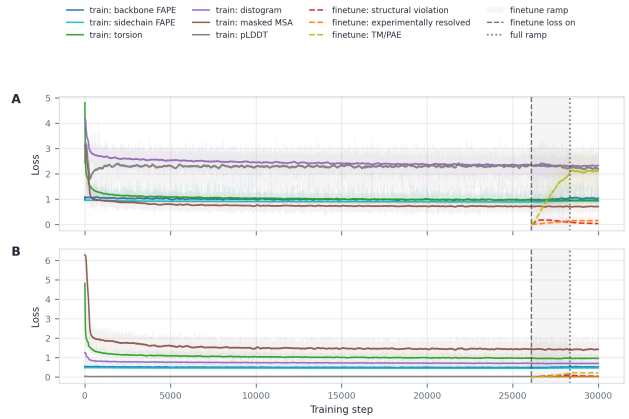


(a) Batchwise 90/10

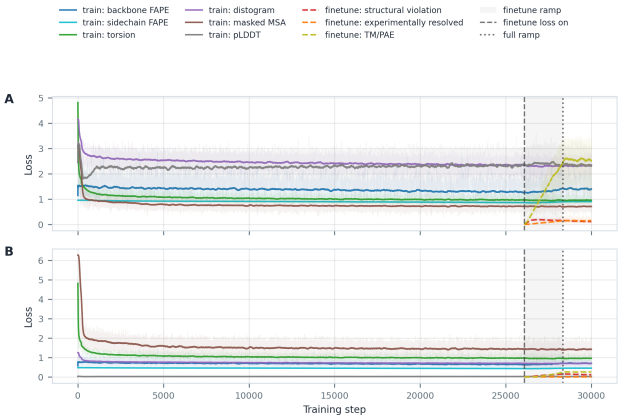
(b) Soft mix 90/10

Figure 12. **Loss-component trajectories for backbone-FAPE clamping policies.** In each subfigure, Panel A shows raw loss components and Panel B shows weighted contributions. The batchwise and soft-mix panels compare stochastic and deterministic implementations of the 90/10 clamping policy.

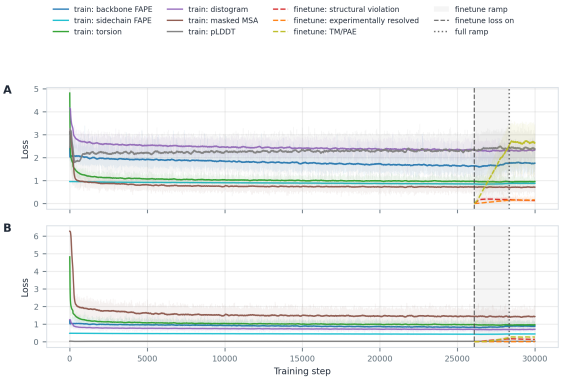
The remaining panels separate samplewise clamping probabilities from fully unclamped schedules, including unclamping only after the fine-tuning boundary.



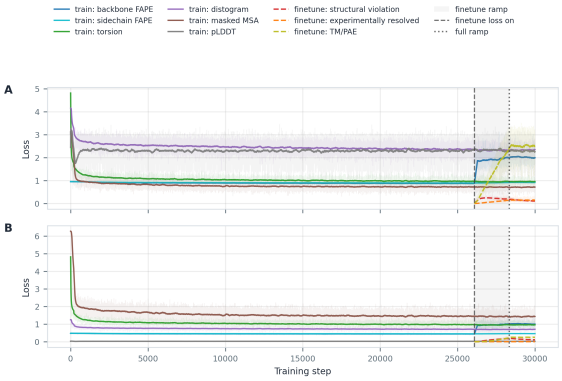
(c) Samplewise 90/10



(d) Samplewise 50/50



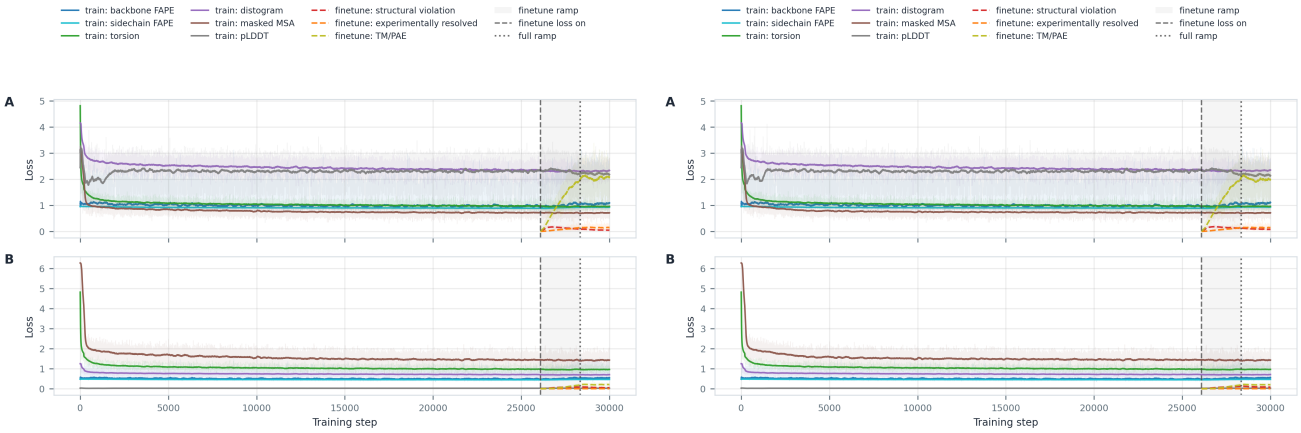
(e) Fully unclamped



(f) Unclamp at fine-tune

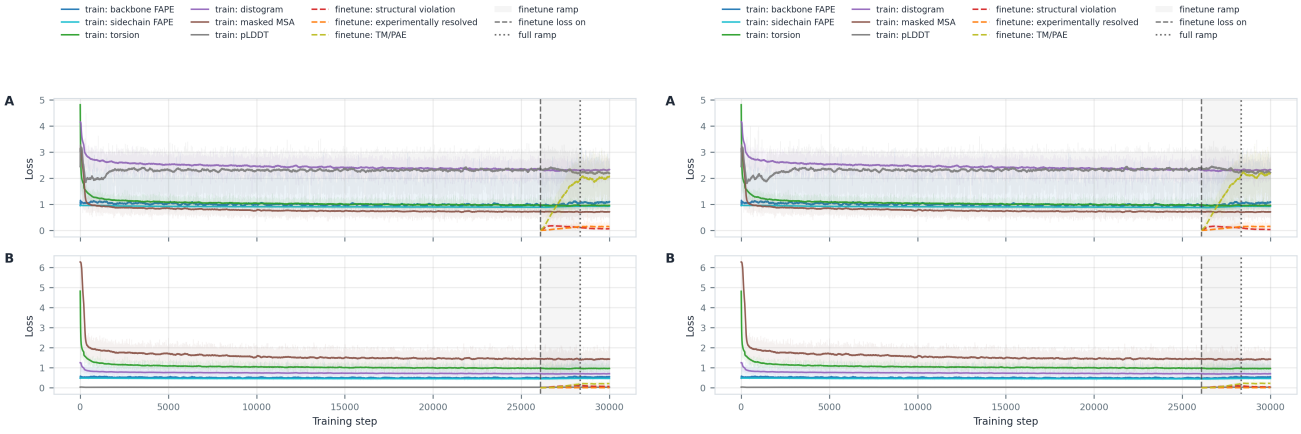
Figure 12. **Loss-component trajectories for backbone-FAPE clamping policies (continued).** These panels show how samplewise and fully unclamped policies change the supervised objective components underlying the validation curves in Figures 9 and 5.

The recycling sweep provides the corresponding component-level comparison for the sampled and fixed recycling policies.



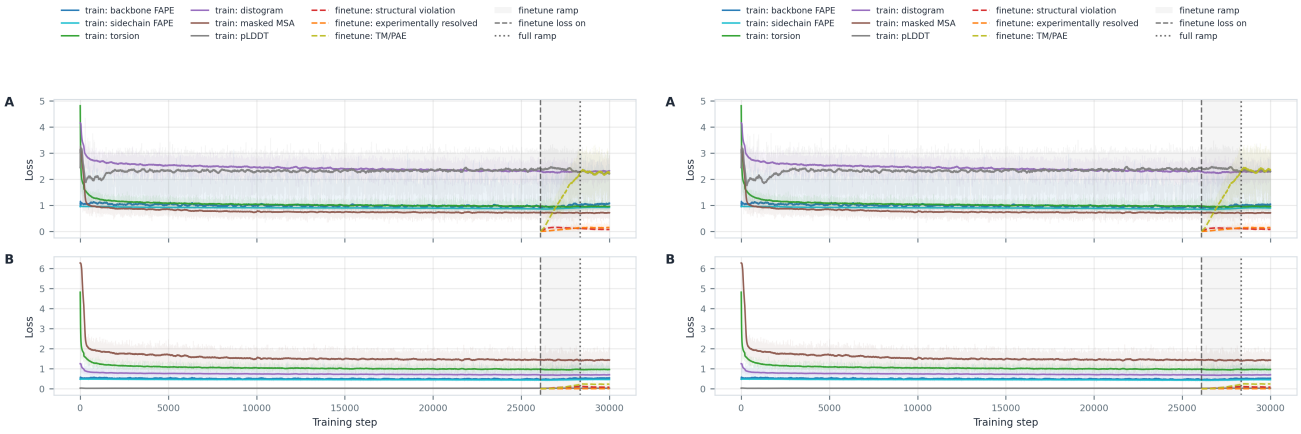
(a) Sampled 1–2 recycles

(b) One cycle



(c) Sampled 1–4 recycles

(d) Sampled 1–6 recycles

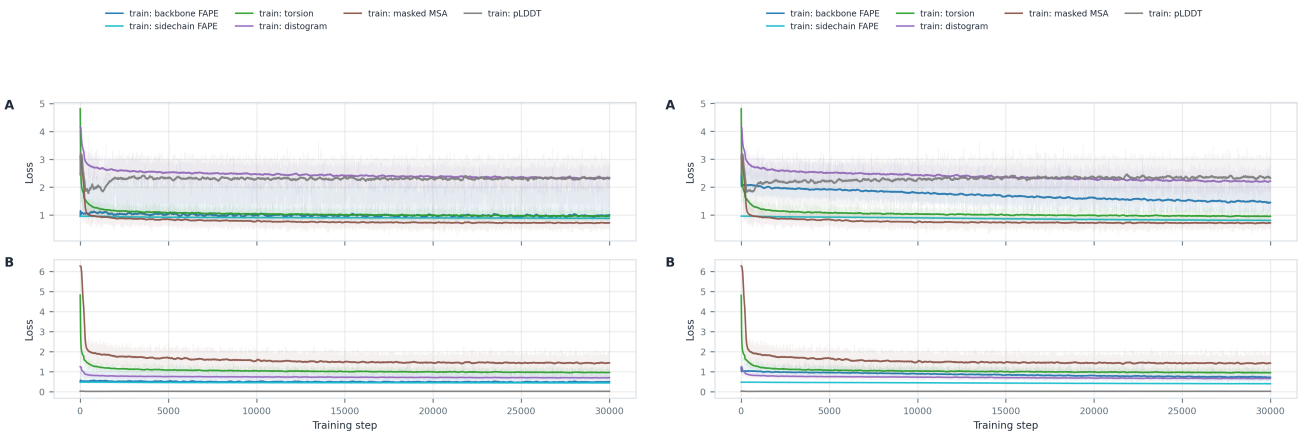


(e) Sampled 1–8 recycles

(f) Fixed 4 recycles

Figure 13. **Loss-component trajectories for recycling-policy ablations.** In each subfigure, Panel A shows raw loss components and Panel B shows weighted contributions. The loss traces complement the validation lDDT-Cα curves by showing the supervised terms for the same medium-model recycling sweep. The "Sampled 1–2 recycles" panel is the medium AF2-default run and reuses the same plot as the corresponding panel of Figure 11.

Finally, the schedule comparison shows the loss traces for removing the fine-tuning phase alone and for combining that schedule with fixed-four recycling and fully unclamped backbone FAPE.



(a) No fine-tune loss

(b) Fixed 4 + unclamped + no fine-tune

Figure 14. **Loss-component trajectories for the loss-schedule and combined ablations.** In each subfigure, Panel A shows raw loss components and Panel B shows weighted contributions. The no-fine-tune run keeps the initial objective active for all 30,000 optimizer steps, while the combined run pairs that schedule with fixed-four recycling and fully unclamped backbone FAPE.

A.5. Qualitative structure examples

The remaining appendix structure figure uses the same public-validation rendering, Kabsch alignment, and color convention as Figure 4 in Section 5.1.

RMSD-selected triptych. The triptych in Figure 15 selects three public-validation chains using the original comparison of aligned Cα RMSD for the tiny and medium models: one where both have comparatively low RMSD, one where the medium model has low RMSD and the tiny model has high RMSD, and one where both have high RMSD. The large prediction is overlaid on those same selected chains for scale comparison. The selected chains are 2k6i_A, 2v66_E, and 4abx_D.

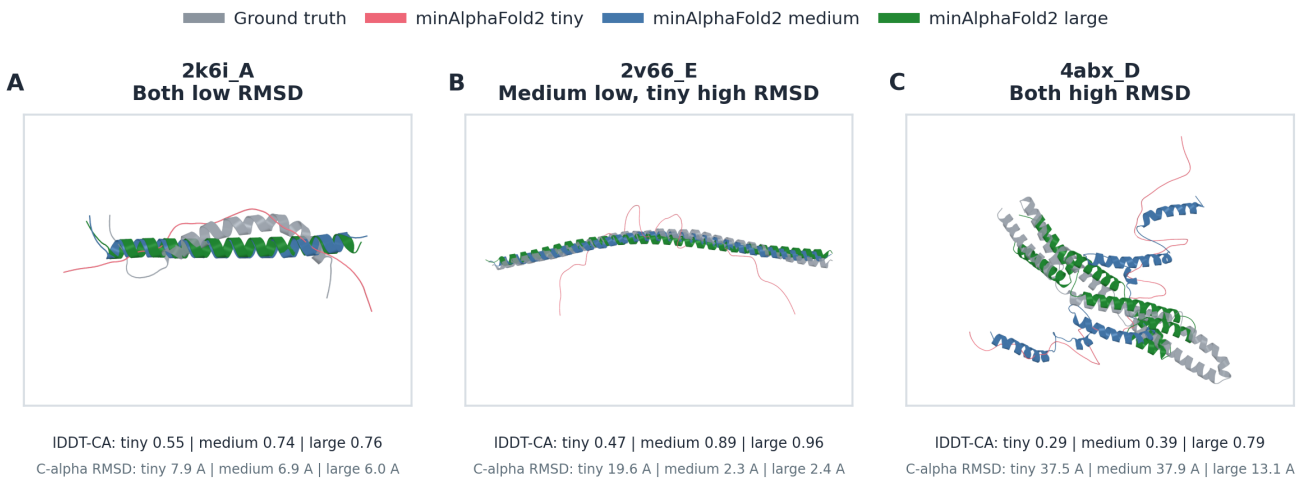


Figure 15. **RMSD-selected public-validation structure examples.** Ground truth, tiny prediction, medium prediction, and large prediction are aligned on valid Cα atoms.

A.6. Compute and artifact accounting

The short-budget calibration and ablation runs in Table 9 were executed on RunPod pods with NVIDIA H100 80-GB HBM3 GPUs, PyTorch 2.8.0+cu128, CUDA 12.8, and cuDNN 9.10.2. The 16 run executions consumed 398.9 measured

H100-hours. At the observed price of $2.99 per H100-hour, these runs cost approximately $1,193. Additional exploratory runs, including full-scale restarts, SimplexFold, Muon, interaction ablations, and legacy controls, were excluded from the core figures. Across complete tracked exploratory runs, measured wall time summed to approximately 375.1 H100-hours.

The largest individual short-budget run was the large scaling run at 137.25 H100-hours, which cost $410.38. The largest individual medium-profile run was the sampled 1–8 recycling experiment at 23.47 H100-hours, which cost $70.18. The shortest run was the tiny scaling run at 10.02 H100-hours, which cost $29.96.

A.7. Goal Mode refines a human-seeded SimplexFold architecture

Beyond controlled ablations, NanoFold's short feedback loop supports long-running, agent-driven model search. Prior work has used agents and language models for machine-learning experimentation, architecture, and program search (Huang et al., 2024; Chen et al., 2023; Romera-Paredes et al., 2024; Lu et al., 2024; Yamada et al., 2025). We ask whether an agent can implement and refine a high-level, human-seeded protein architecture hypothesis within a fixed experimental regime. We used GPT-5.5 in the Codex harness, beginning from the AF2-medium NanoFold baseline and supplying the higher-order SimplexFold research direction described below. The initial task prompt targeted public-validation lDDT-Cα above 0.7 while keeping the parameter count within 5% of AF2-medium. For the analysis reported here, scored ledger entries are ordered by experiment number and evaluated by final-or-stop public-validation FoldScore; the lineage and headline comparison therefore use FoldScore. The ledger uses experiment identifiers through E151, and its scored entries have heterogeneous stopping budgets. E151 is shown over its 30,000-step logged trajectory. Evaluating the requested 5% parameter-match target requires a final E151 parameter-count measurement, which is absent from the current artifact set. The full prompt and persistence protocol appear in Section A.8.

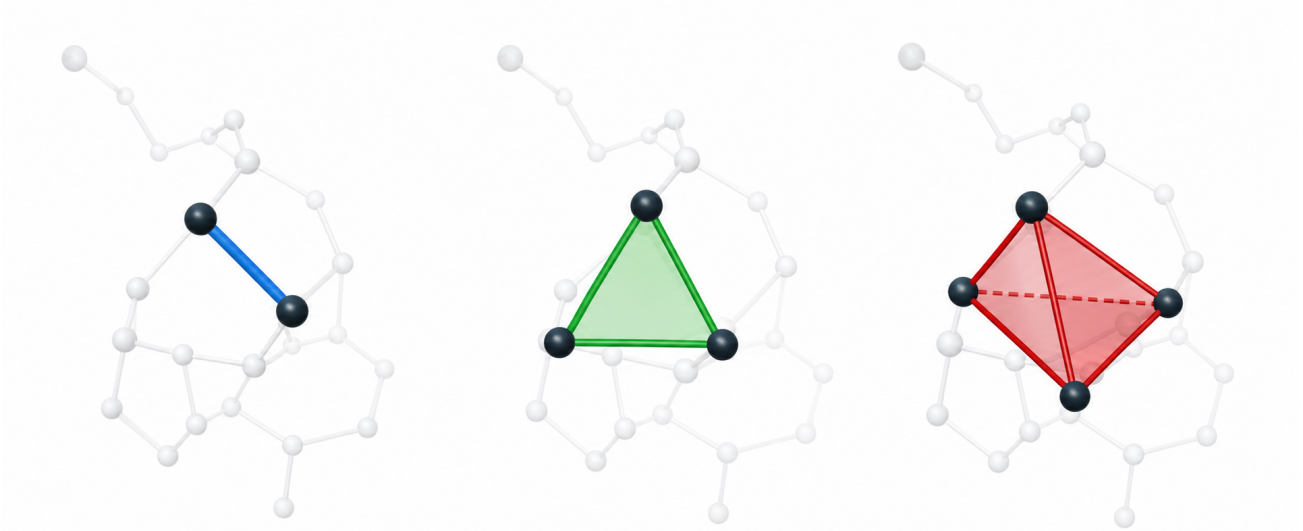


Figure 16. **SimplexFold augments pair-indexed protein representations with persistent higher-order states.** The highlighted edge, triangular face, and tetrahedral cell illustrate the progression from 1-simplices between residue pairs to 2- and 3-simplices over residue triples and quadruples. SimplexFold uses selected higher-order objects as explicit learned states for modeling local protein geometry alongside the AF2-style pair representation.

AlphaFold2 already performs triadic computation: its Evoformer maintains a persistent pair-indexed state and uses triangle multiplication and triangle attention over a third residue to update that state (Jumper et al., 2021). SimplexFold investigates a different representational choice, motivated by prior hypergraph and simplicial networks (Feng et al., 2019; Bodnar et al., 2021; Eijkelboom et al., 2023; Battiloro et al., 2025): it maintains additional learned states indexed directly by selected residue triples and quadruples and exchanges information across pair-, triangle-, and tetrahedron-indexed states (Figure 16).

We placed the agent in an iterative Goal Mode loop. Initial high-level prompts to improve AF2-medium mostly produced changes to loss weights and optimizer hyperparameters, so we supplied the SimplexFold direction as a human-authored research hypothesis and asked the agent to implement and refine it. Three persistent files stored the plan, experiment index, and running observations across the campaign; Section A.8 gives their exact names and reconciles them with those requested

in the initial prompt.

The campaign ledger uses experiment identifiers through E151, with scored entries that could terminate at different budgets. Figure 17 therefore reports each scored entry's final-or-stop FoldScore. Persistent state allowed the agent to revisit recorded configurations, continue promising branches, and discard weaker variants. The figure summarizes the trajectory and the lineage that culminated in E151, the highest-FoldScore public-validation configuration in the ledger. Repeated public-validation selection makes E151 a development result; reusable-holdout and leaderboard analyses motivate a one-shot sealed evaluation of the frozen configuration for confirmatory ranking (Dwork et al., 2015; Blum & Hardt, 2015).

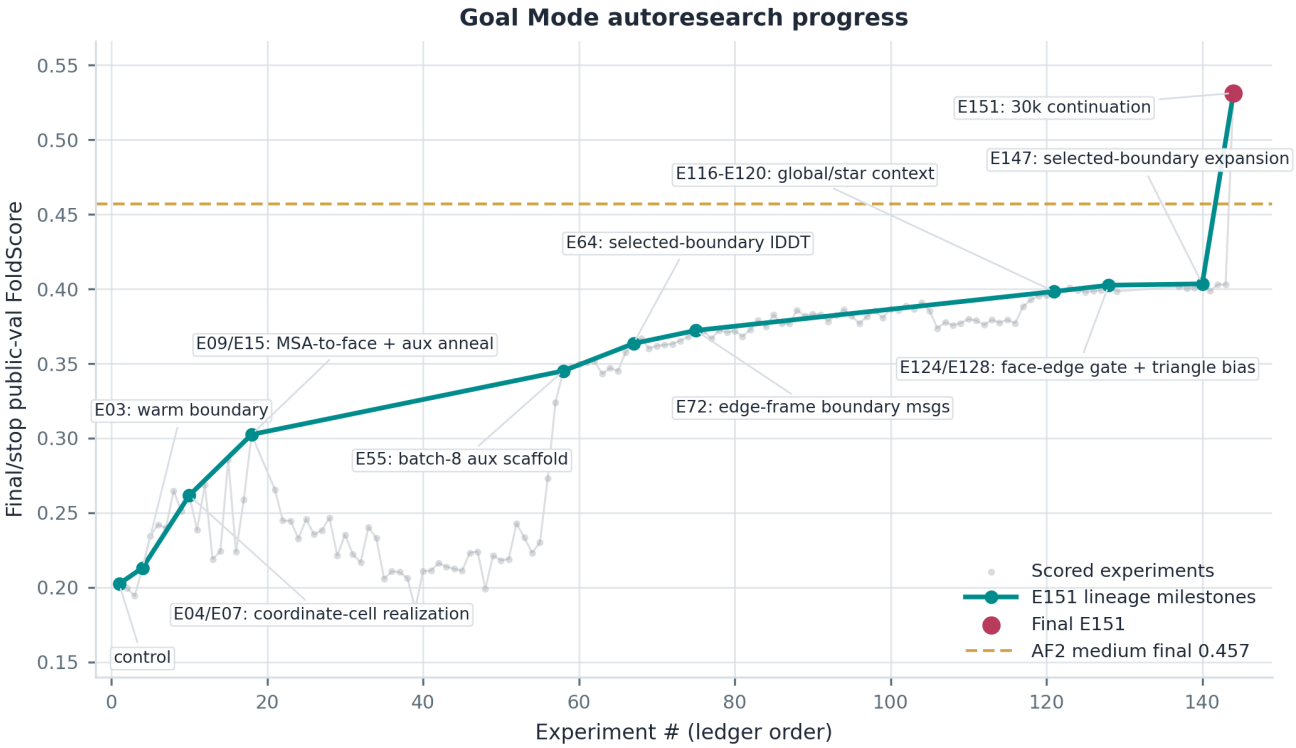


Figure 17. **Goal Mode development lineage.** The gray trace shows the final-or-stop public-validation FoldScore of each scored ledger entry; these points have heterogeneous stopping budgets. The teal path follows the lineage leading to E151 across changes to boundary construction, simplex and coordinate representations, MSA-to-face pathways, auxiliary losses, context and geometric message paths, and longer training continuations. The dashed line marks the AF2-medium final public-validation FoldScore (0.457).

The complete trajectory is deliberately irregular: most proposed configurations did not establish a new best-so-far result, and several promising directions produced gains only after being combined with changes identified in earlier runs. The best-so-far lineage nevertheless shows sustained progress from the initial SimplexFold implementation through changes to boundary handling, coordinate and simplex representations, geometric updates, and continued training. This progression culminated in E151, the highest-scoring configuration in the campaign ledger. We next compare that model directly with the AF2-medium baseline under the same NanoFold public-validation setting.

On the adaptively reused public-validation split, E151 reaches a FoldScore of 0.533 at 30,000 logged steps, compared with 0.457 for AF2-medium and 0.504 for the strongest combined recipe ablation (Figure 18), making it the strongest development result reported here. Its configuration combines higher-order states with other architectural and training changes, so the measured improvement applies to the complete bundle. Component ablations can quantify the simplex-state contribution, and a one-shot sealed evaluation can establish a confirmatory ranking.

Taken together, these experiments demonstrate NanoFold's support for sustained, agent-driven implementation and refinement across a long sequence of model-development decisions. This case study evaluates long-horizon execution of an author-supplied hypothesis within a fixed biological modeling environment; controlled comparison with alternative search algorithms is a separate experiment. It complements benchmark-based evaluations of research agents (Huang et al., 2024; Chen et al., 2025).

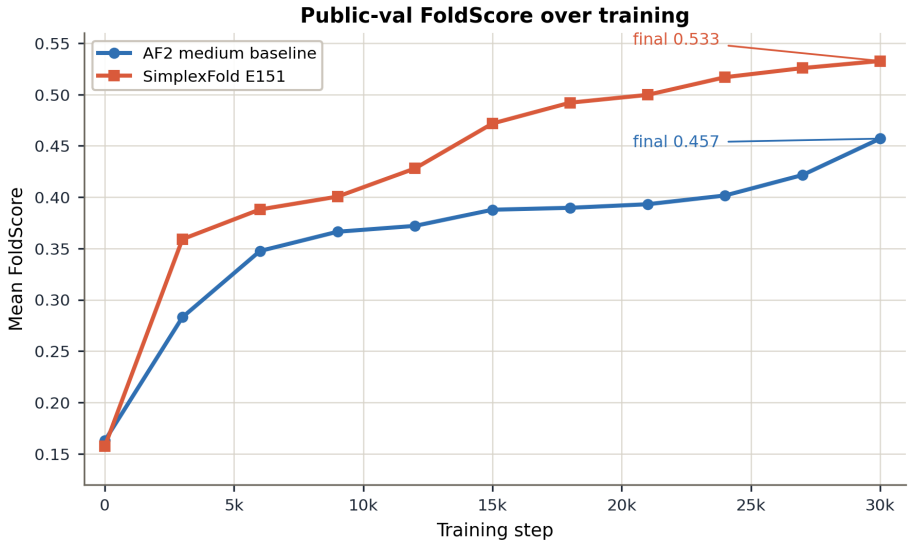


Figure 18. **Goal Mode refines human-seeded SimplexFold on the public-validation split.** A longer-running Codex Goal Mode loop used NanoFold to iterate over SimplexFold variants, continue promising branches, and reject weak ones. The final E151 SimplexFold public-validation curve reaches 0.533 FoldScore at 30,000 logged steps, compared with 0.457 for the AF2-medium baseline under the same NanoFold public-validation setting.

A.8. Goal Mode protocol and reporting semantics

A.8.1. Human-supplied task and agent scope

The SimplexFold research direction, baseline, objective, and constraints were supplied by the authors. GPT-5.5 received the following initial Goal Mode prompt in the Codex harness:

> `/goal` *I'd like you to iterate on SimplexFold (parameter matched to AF2 medium) on the NanoFold dataset. We should aim for a validation lddt Calpha score that exceeds 0.7 while keeping parameters within 5% of the AF2 medium model. You can make any changes you'd like to the loss functions or architecture, but keep all changes in the spirit of the README in the SimplexFold repo. Record your high level plans in* `PLAN.md`*, all of your experiment ideas in* `EXPERIMENTS.md`*, and keep a running log of notes in* `EXPERIMENTS_NOTES.md`

The agent then implemented candidate changes, launched and monitored training, read public-validation results, and planned follow-up configurations. The campaign therefore evaluates agent implementation and refinement of the author-supplied SimplexFold direction.

A.8.2. Persistent state and experiment counting

Although the initial prompt requested `PLAN.md`, `EXPERIMENTS.md`, and `EXPERIMENTS_NOTES.md`, the implemented loop used the following three persistent files:

1. `PLAN.md` stored the instructions and current high-level plan.
2. `EXPERIMENT_INDEX.md` indexed result directories and reusable scripts.
3. `RUNNING_NOTES.md` recorded candidate experiments, run status, and observations.

The campaign used this three-file persistent-state design throughout. A memory-design ablation would measure its incremental contribution.

The ledger uses identifiers through E151 and heterogeneous stopping budgets; Figure 17 reports each scored entry's final-or-stop public-validation FoldScore. Although the prompt targeted lDDT-Cα, the reconstructed campaign analysis uses FoldScore, making E151 a FoldScore development result. The archive contains its 30,000-step public-validation trajectory; parameter matching and sealed-split performance require new measurements.